

Deep Learning for Computer Vision

114-1/Fall 2025

<https://cool.ntu.edu.tw/courses/53063> (NTU COOL)

<http://vllab.ee.ntu.edu.tw/dlcv.html> (Public website)

Yu-Chiang Frank Wang 王鈺強, Professor

Dept. Electrical Engineering, National Taiwan University

What to Cover Today...

- Image Segmentation
- Object Detection
 - Detection via Sliding Windows
 - Two-Stage vs. Single-Stage Detectors
 - ~~Transformer-based Detectors~~
 - ~~3D Detection~~

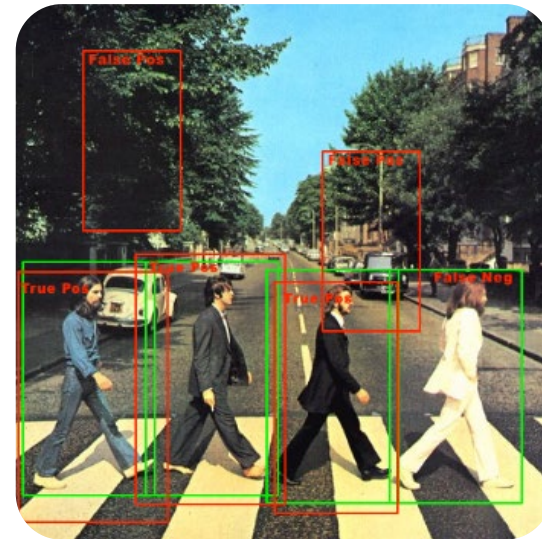
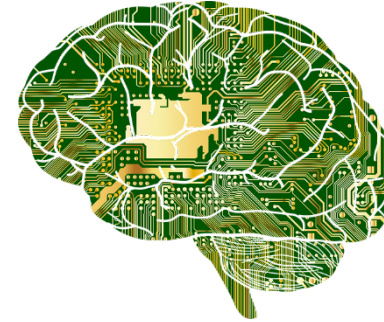
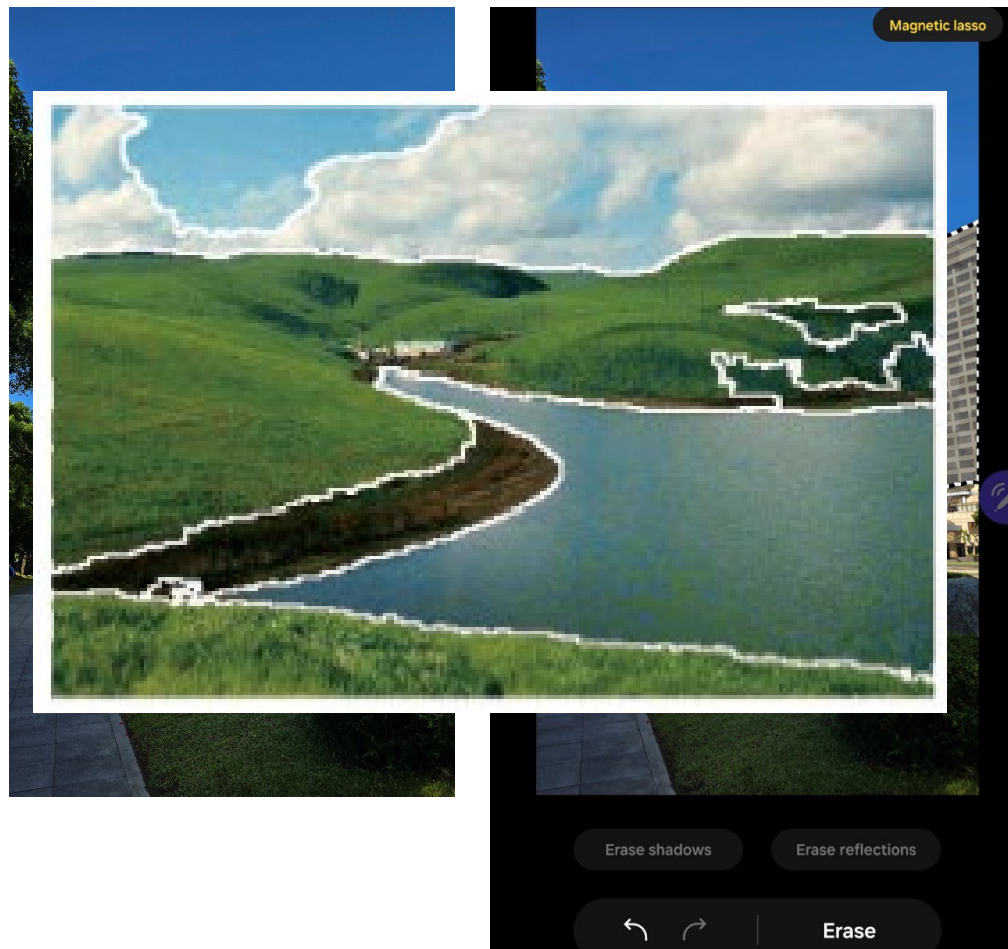
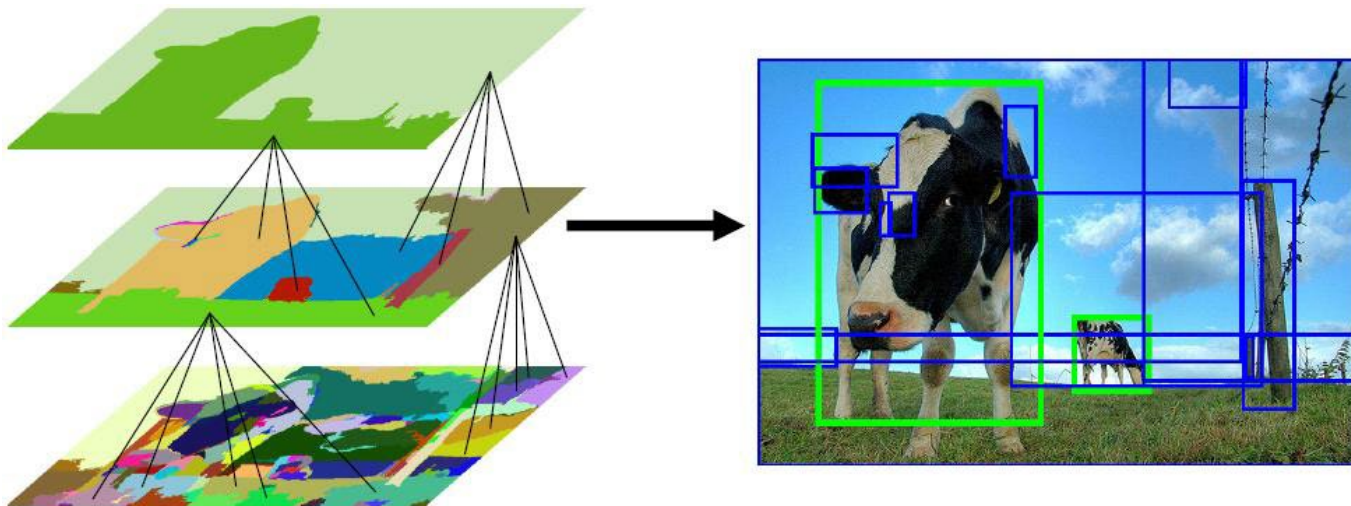


Image Segmentation

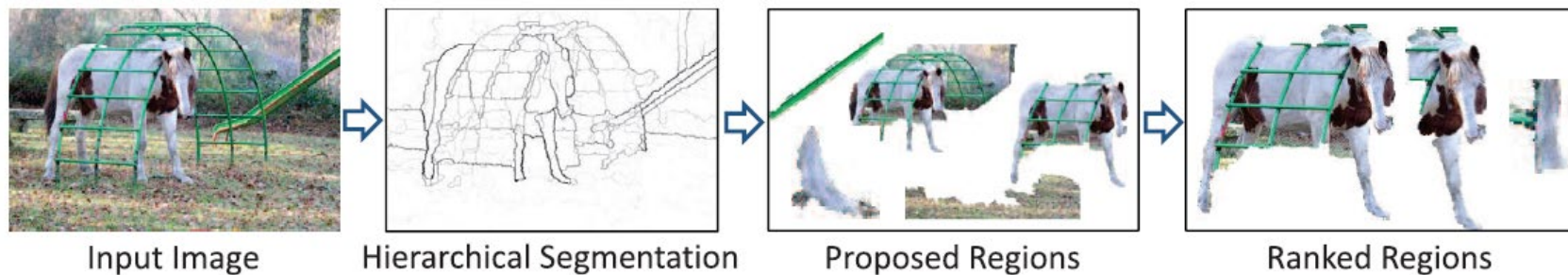
- Goal: Group pixels into meaningful or perceptually similar regions
- Any recent smart phone applications?



Segmentation for Object Proposal



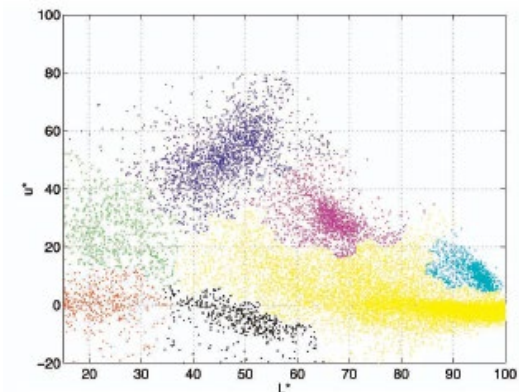
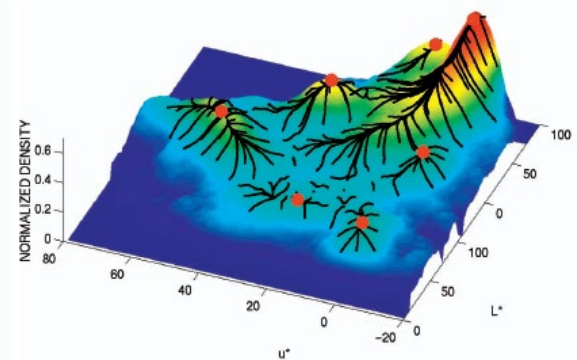
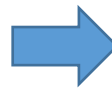
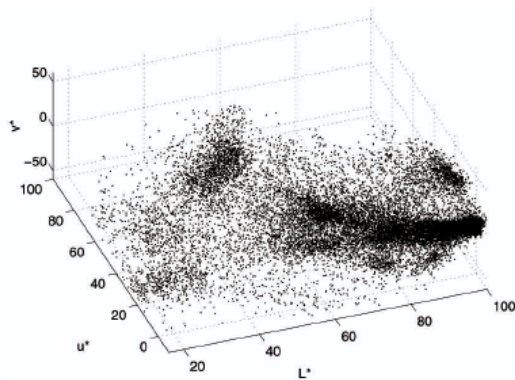
“Selective Search” [Sande, Uijlings et al. ICCV 2011, IJCV 2013]



[Endres Hoiem ECCV 2010, IJCV 2014]

Segmentation via Clustering – Unsupervised Learning based Approaches

- K-means clustering -> [R, G, B, x, y] as pixel features
- Mean-shift
 - Find modes of the following non-parametric density



Superpixels

- A relatively simpler task of image segmentation
- Divide an image into a large number of image regions, such that each region lies within object boundaries.
- Examples
 - Watershed
 - Felzenszwalb and Huttenlocher graph-based
 - Turbopixels
 - SLIC



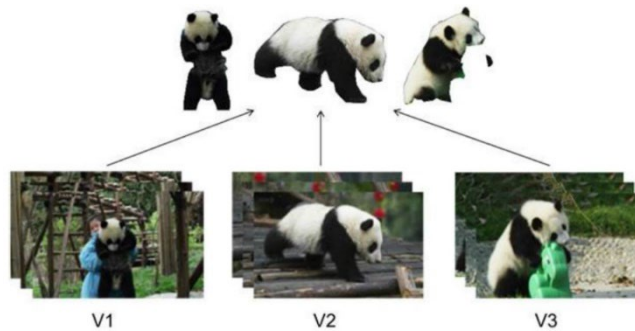
Semantic Segmentation – Supervised Learning based Approaches

- Semantic Segmentation
 - Assign a class label to each pixel in the input image (i.e., [pixel-level classification](#))
 - Not like instance segmentation, do not differentiate instances; only care about pixel labels



More Tasks in Segmentation

- Cosegmentation
 - Segmenting common objects from multiple images
 - Unsup. or supervised? Why preferable?

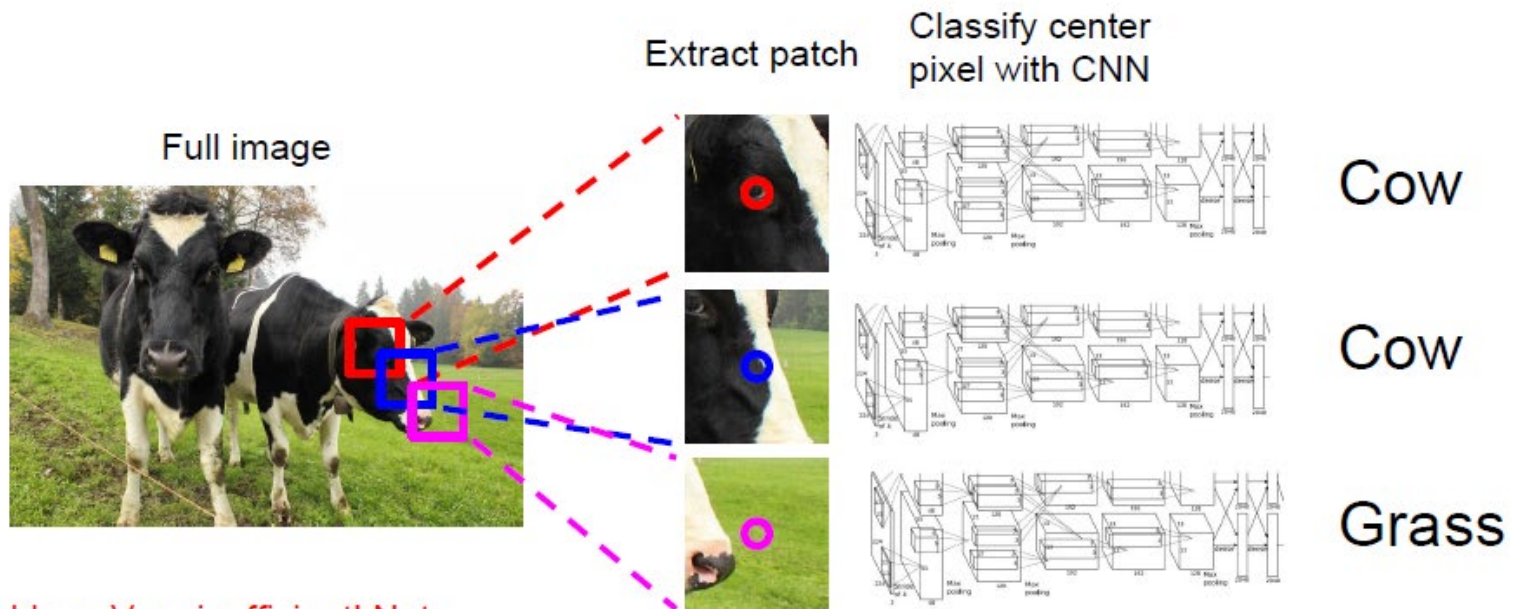


- Instance Segmentation
 - Assign a particular class label for each object instance
 - Unsuper. or supervised?



Semantic Segmentation

- Sliding Window
 - Patch or pixel-level classification
 - Any concern?



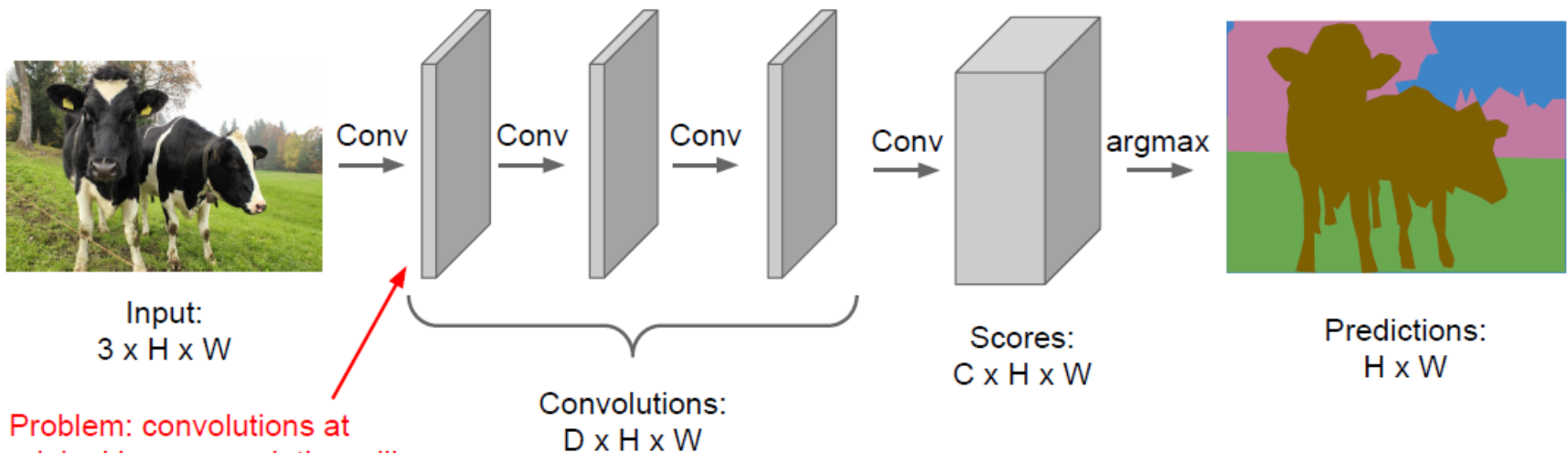
Problem: Very inefficient! Not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Semantic Segmentation

- Fully Convolutional Nets
 - The prediction output is a $H \times W$ map, which can be view as a $C \times H \times W$ class-label matrix.
 - Performing pixel-level classification by mapping the output feature map ($C \times H \times W$) to a class-label matrix ($C \times H \times W$).

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Problem: convolutions at original image resolution will be very expensive ...

Semantic Segmentation

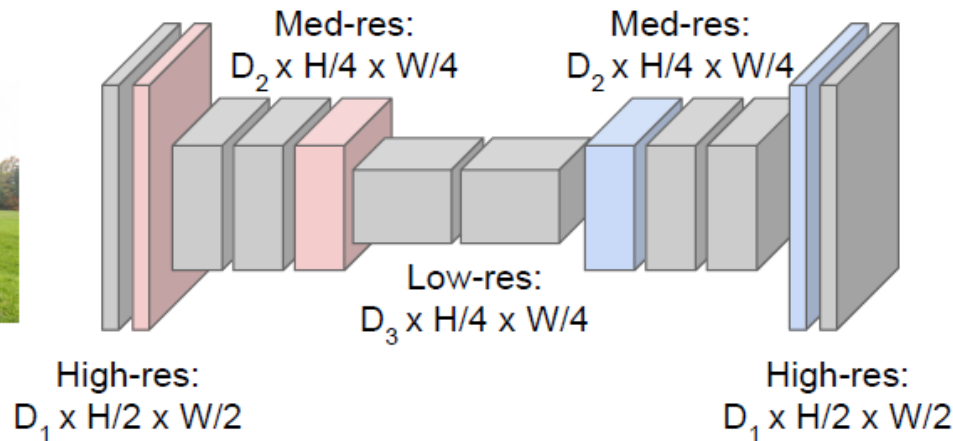
- Fully Convolutional Nets (cont'd)

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
???



Predictions:
 $H \times W$

In-Network Upsampling

- Unpooling

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4



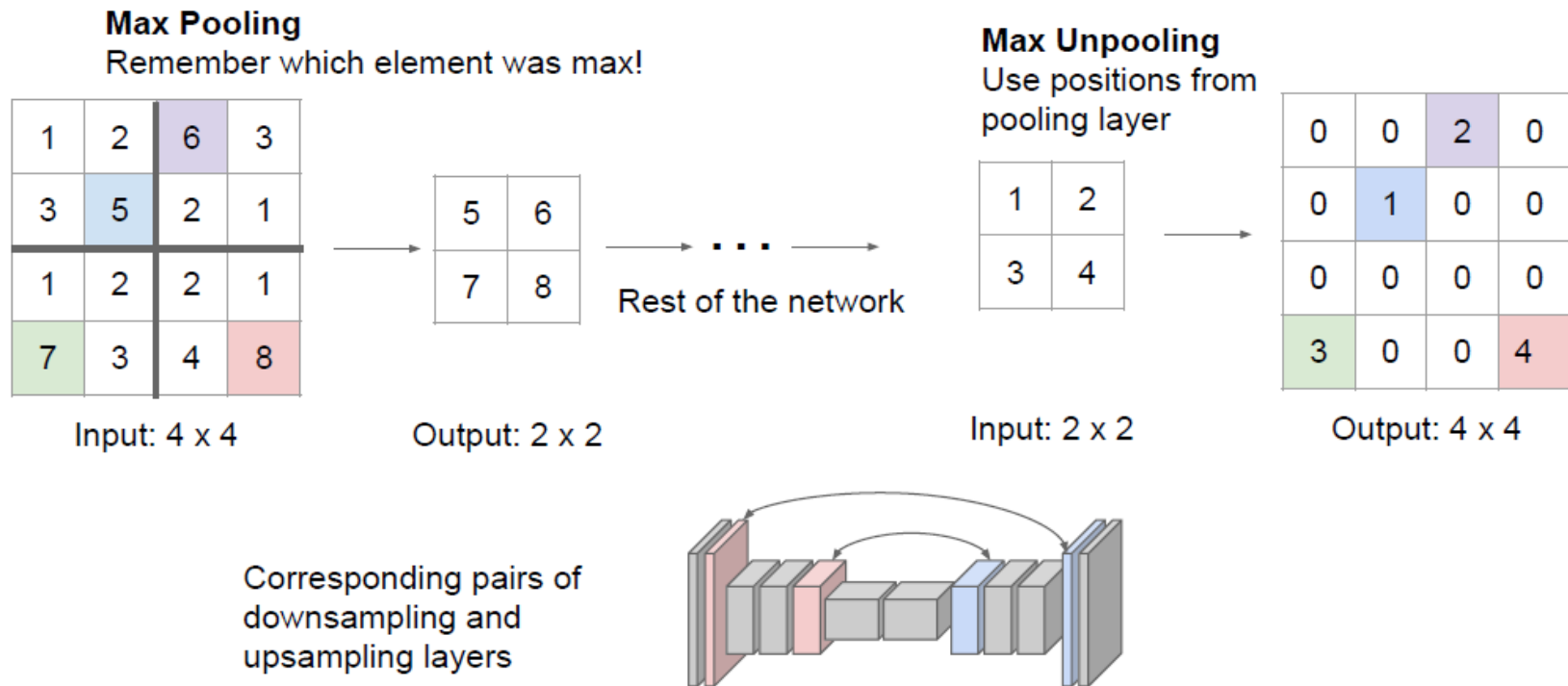
1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

Output: 4 x 4

In-Network Upsampling

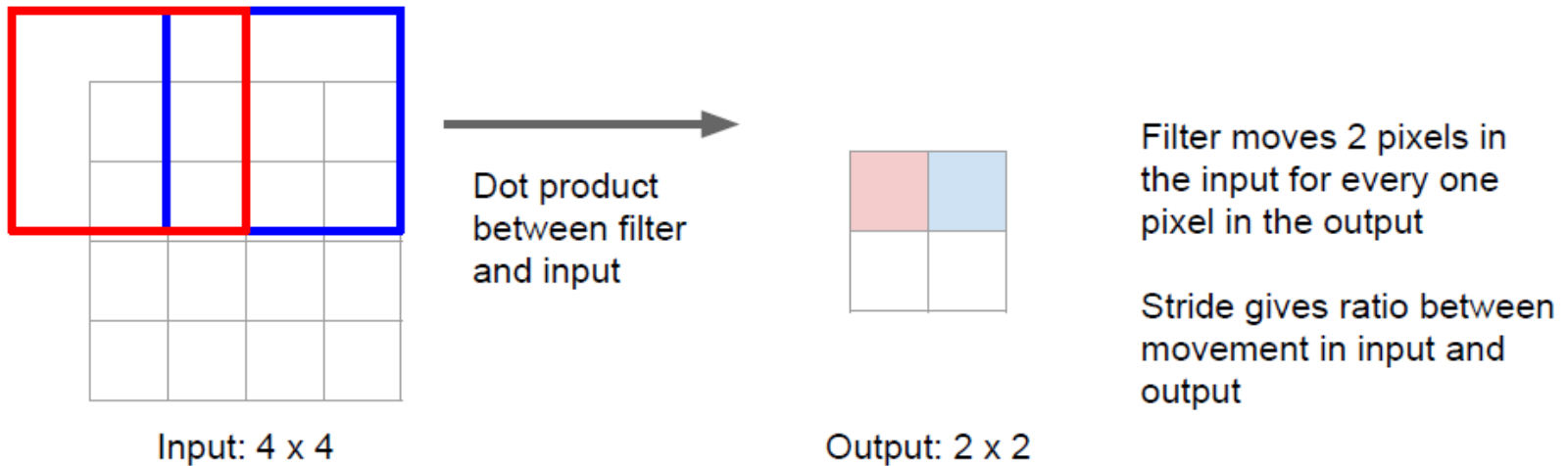
- Max Unpooling
 - What's the price to pay?



In-Network Upsampling

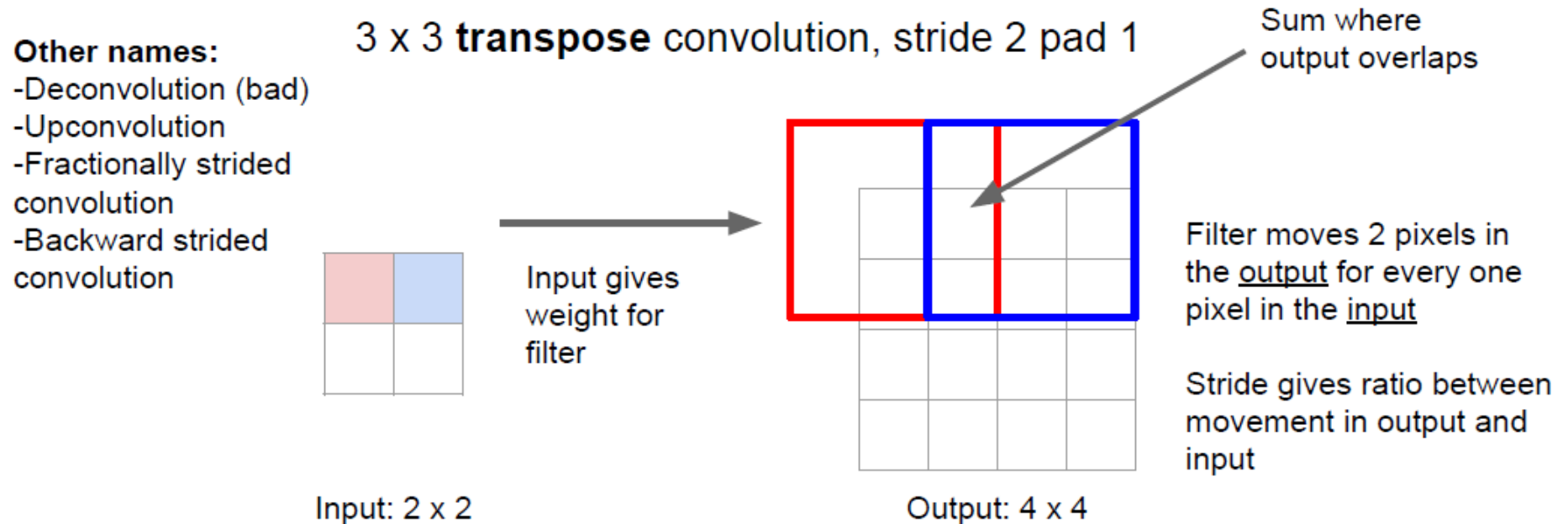
- Learnable Upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 2 pad 1



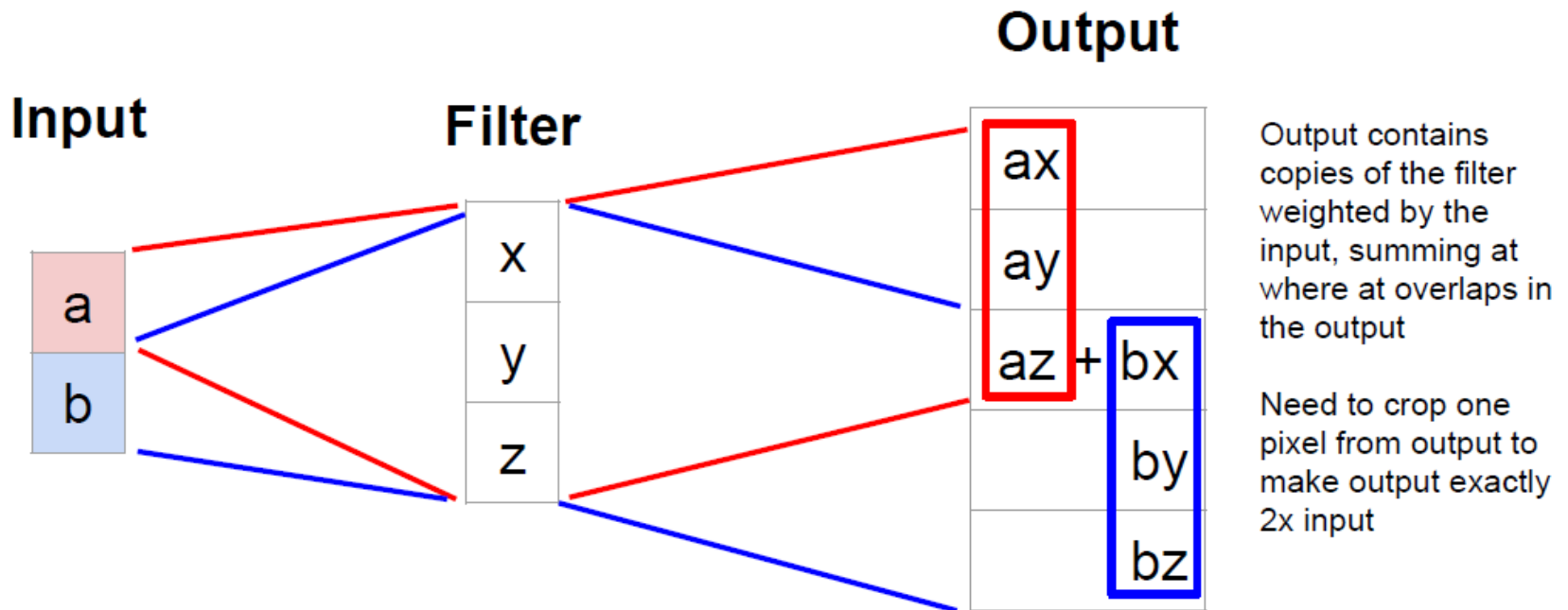
In-Network Upsampling

- *Transpose* Convolution



In-Network Upsampling

- **Transpose Convolution**
 - See a 1D example below:



In-Network Upsampling

- **Transpose Convolution**
 - Example as matrix multiplication

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X \vec{a}$$

$$\begin{bmatrix} x & y & x & 0 & 0 & 0 \\ 0 & x & y & x & 0 & 0 \\ 0 & 0 & x & y & x & 0 \\ 0 & 0 & 0 & x & y & x \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=1, padding=1

Convolution transpose multiplies by the transpose of the same matrix:

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

When stride=1, convolution transpose is just a regular convolution (with different padding rules)

In-Network Upsampling

- **Transpose Convolution**
 - Example as matrix multiplication

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=2, padding=1

Convolution transpose multiplies by the transpose of the same matrix:

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

When stride>1, convolution transpose is no longer a normal convolution!

Fully Convolutional Networks (FCN)

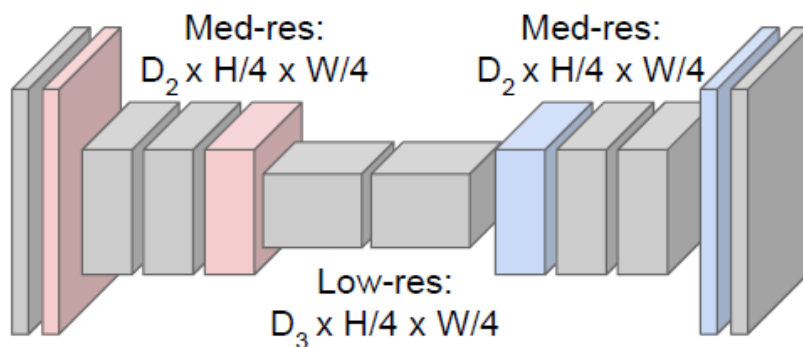
- Remarks
 - All layers are convolutional
 - End-to-end training

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



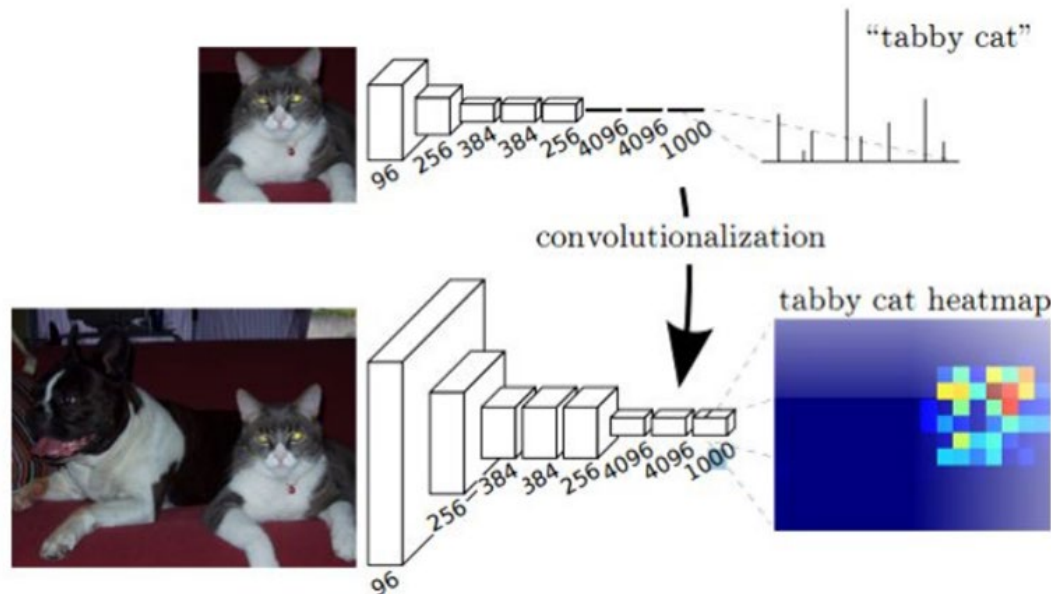
Upsampling:
Unpooling or strided
transpose convolution



Predictions:
 $H \times W$

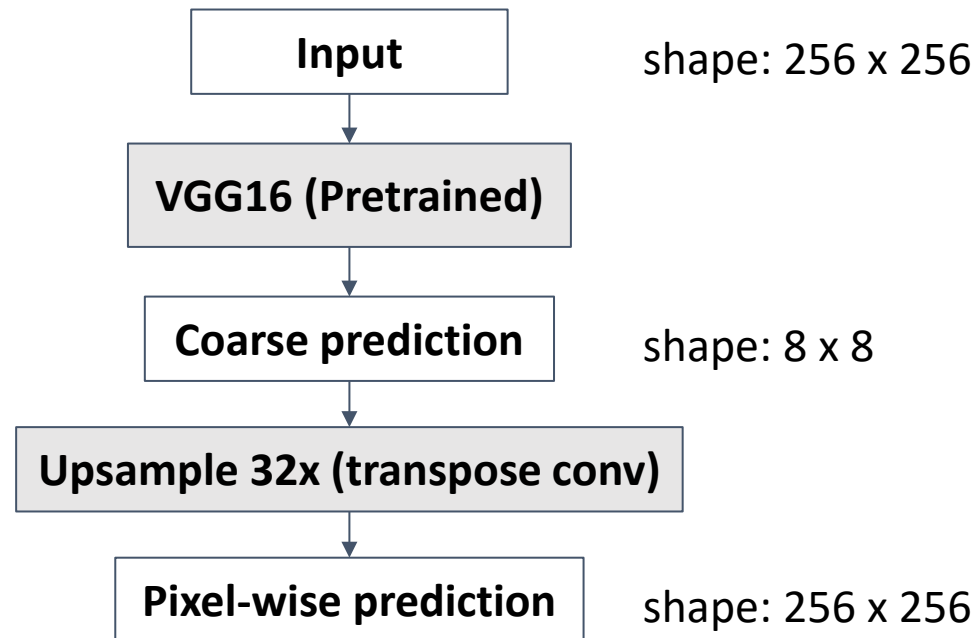
Fully Convolutional Networks (FCN)

- More details
 - Adapt existing classification network to fully convolutional forms
 - Remove **flatten layer** and replace fully connected layers with conv layers
 - Use transpose convolution to upsample pixel-wise classification results



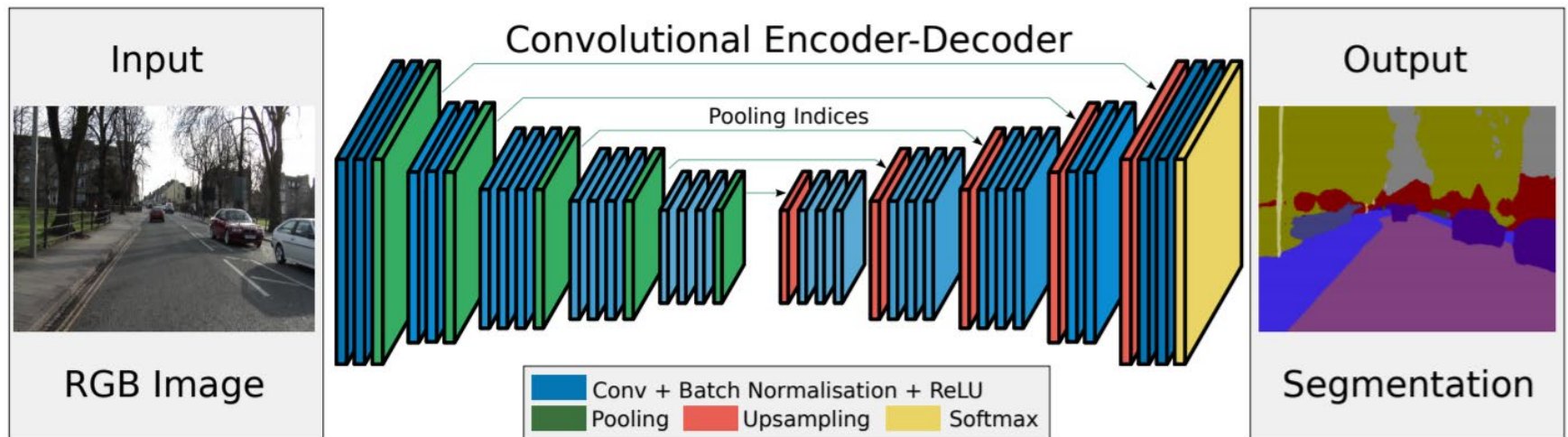
Fully Convolutional Networks (FCN)

- Example
 - **VGG16-FCN32s**
 - Loss: pixel-wise cross-entropy
i.e., compute cross-entropy between each pixel and its label, and average over all of them



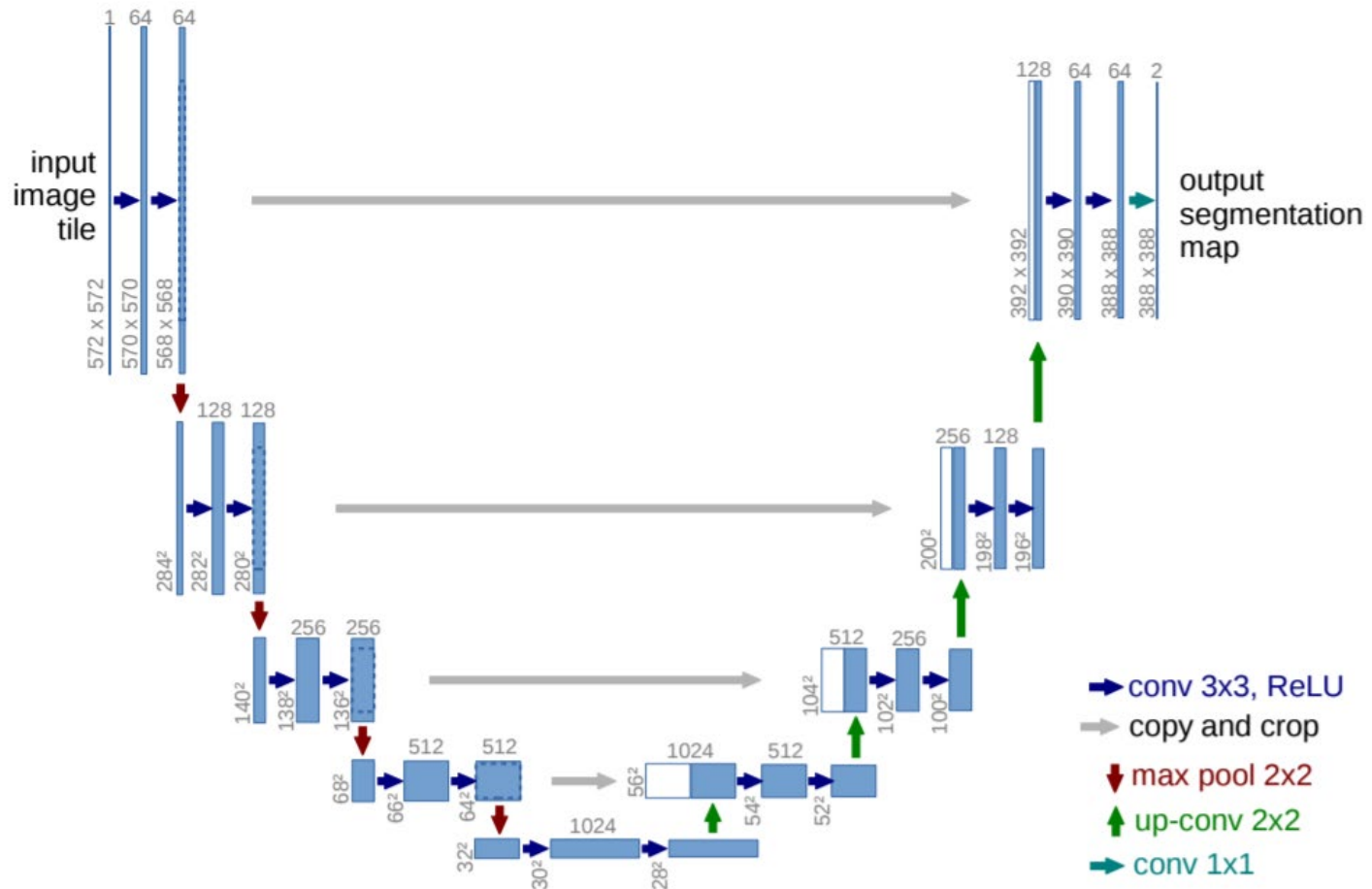
SegNet

- Efficient architecture (memory + computation time)
- Upsampling reusing **max-unpooling indices**
- Reasonable results without performance boosting addition
- Comparable to FCN



“SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation” [\[link\]](#)

U-Net



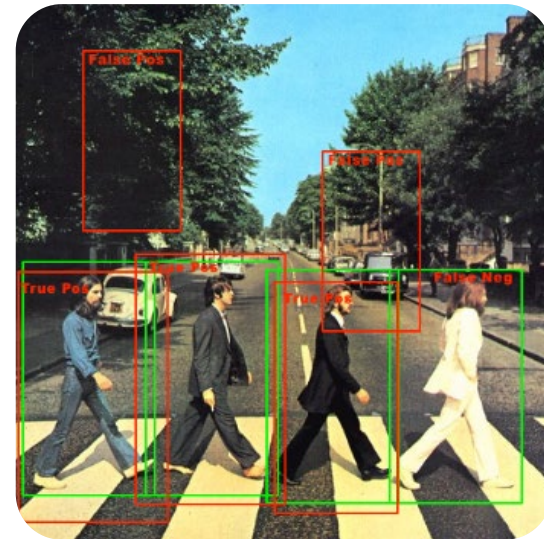
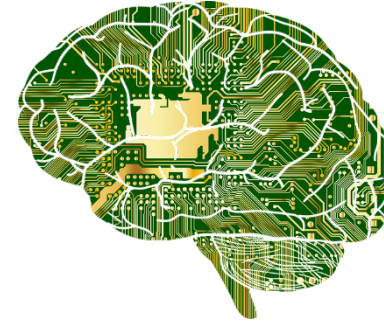
U-Net: Convolutional Networks for Biomedical Image Segmentation [\[link\]](#)

Additional Remarks: Enhanced Spatial Information

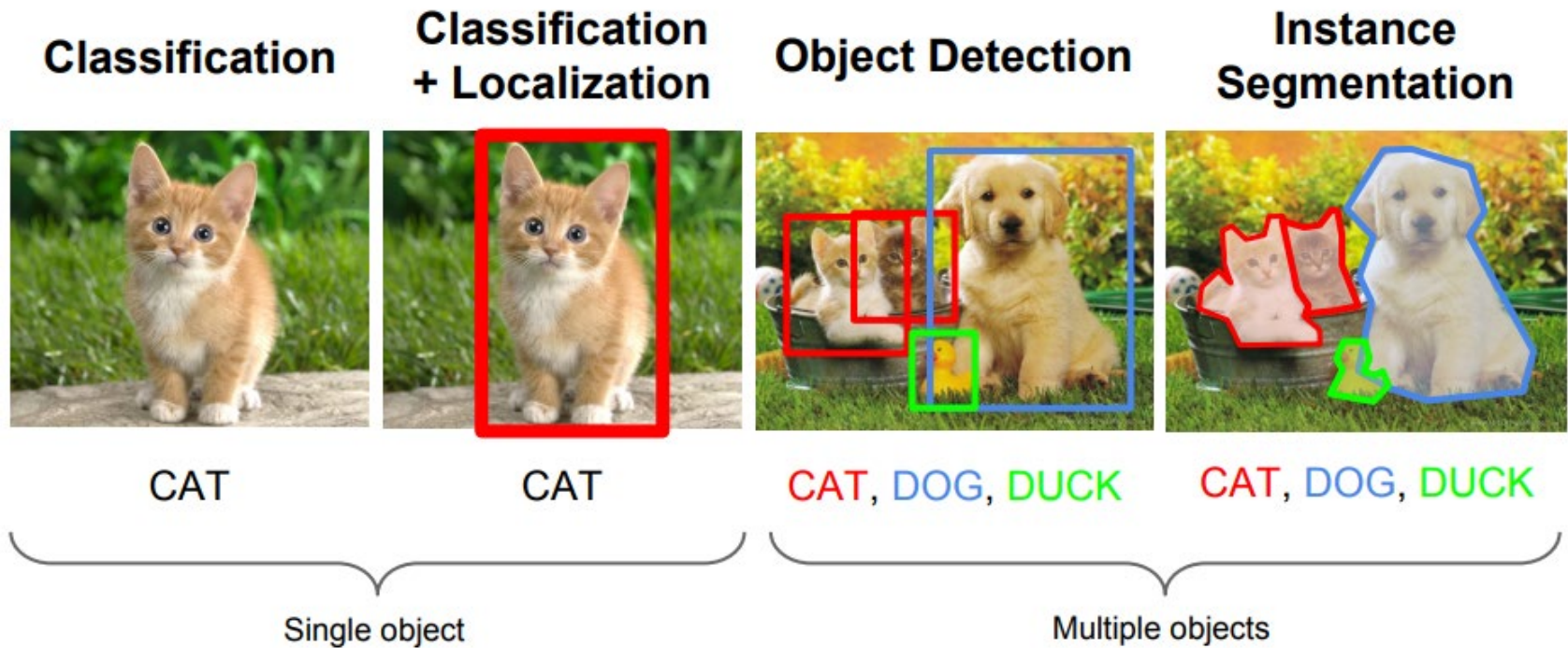
- For semantic segmentation, **spatial information** is of great importance
- It is desirable for the model to observe both the target pixel/region and its **neighboring areas**
 - Atrous (or dilated) convolution
- Features across **different scales** should be considered
 - Spatial pyramid pooling
- **Will comment on this part in future lectures (e.g., object detection)**

What to Cover Today...

- Image Segmentation
- Object Detection
 - Detection via Sliding Windows
 - Two-Stage vs. Single-Stage Detectors
 - ~~Transformer-based Detectors~~
 - ~~3D Detection~~

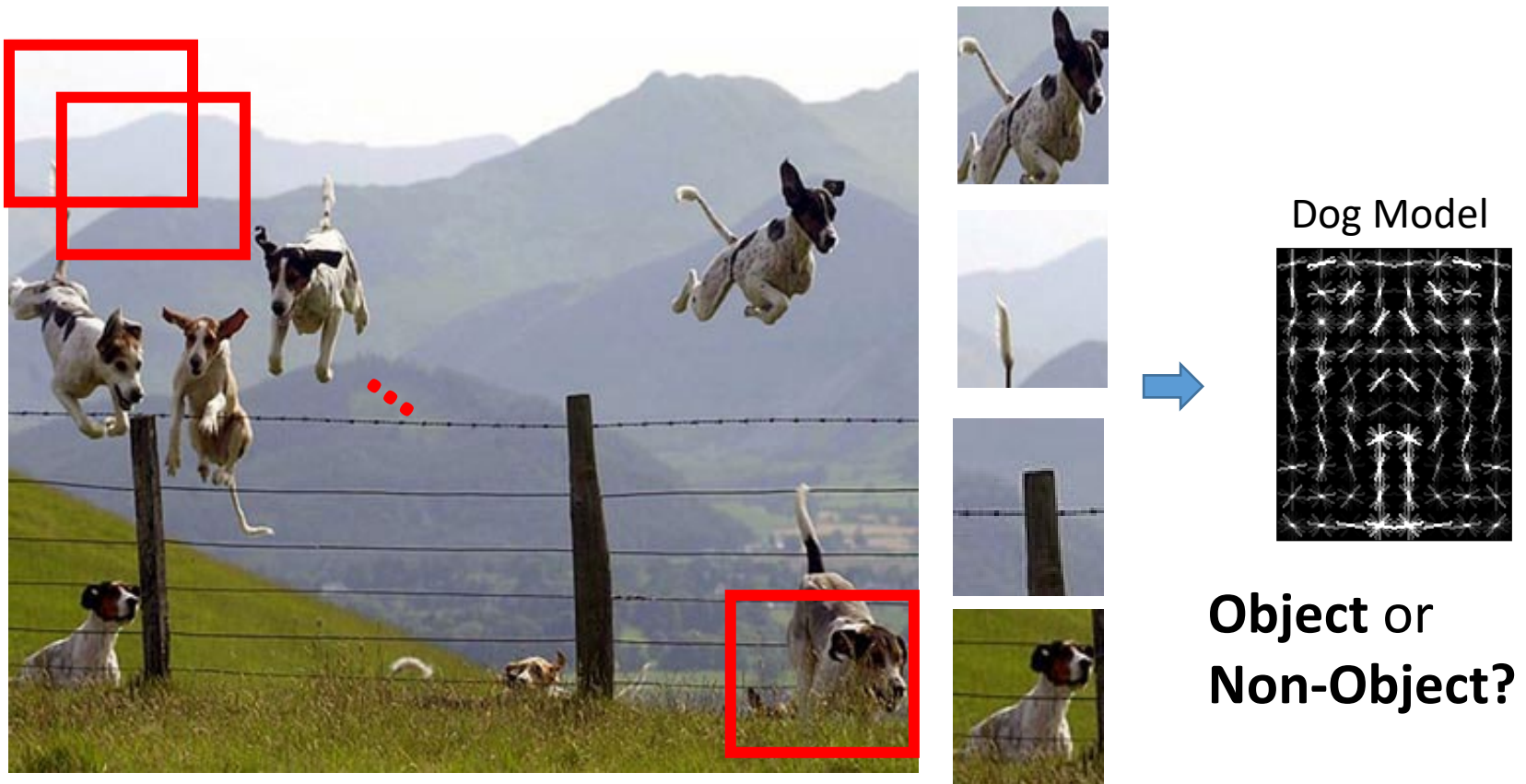


Roadmap



Object Detection

- Focus on object search: “Where is it?”
- Build templates that quickly differentiate object patch from background patch



General Process of Object Recognition

Specify Object Model



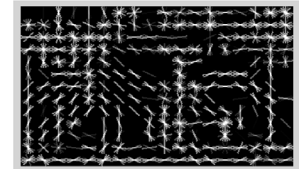
Generate Hypotheses



Score Hypotheses



Resolve Detection



Gradient/region based or CNN features, usually based on summary representation with classification/voting results

Rescore each proposed object based on the entire candidate set

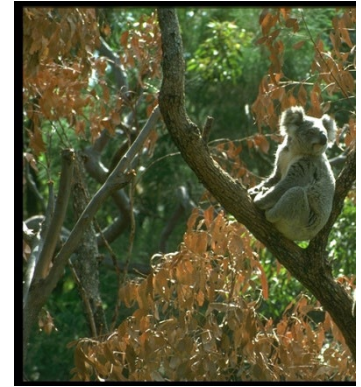
Challenges in Modeling the Object Classes



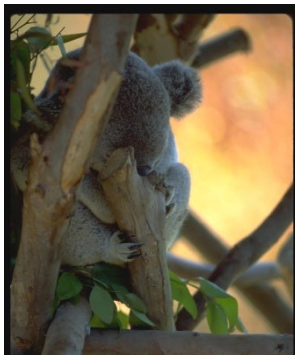
Illumination



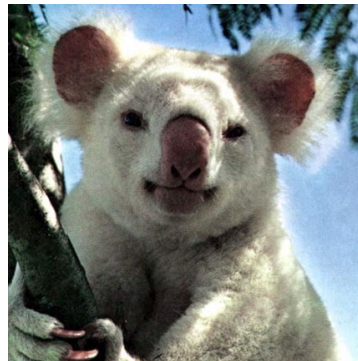
Object pose



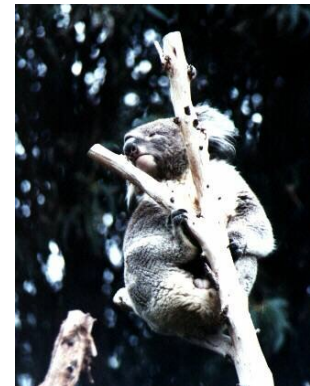
Clutter



Occlusion



Intra-class appearance



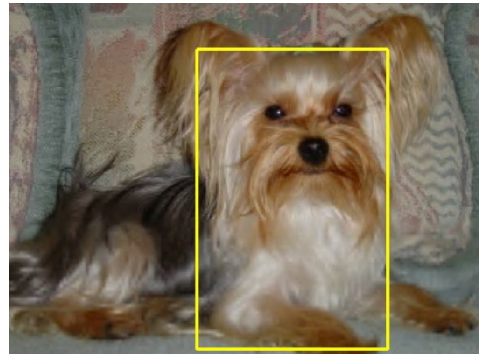
Viewpoint

Challenges in Modeling the Non-object Classes

True
Detection



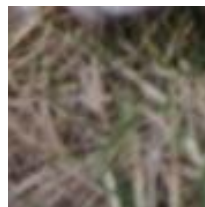
Bad
Localization



Confused with
Similar Object



Misc. Background

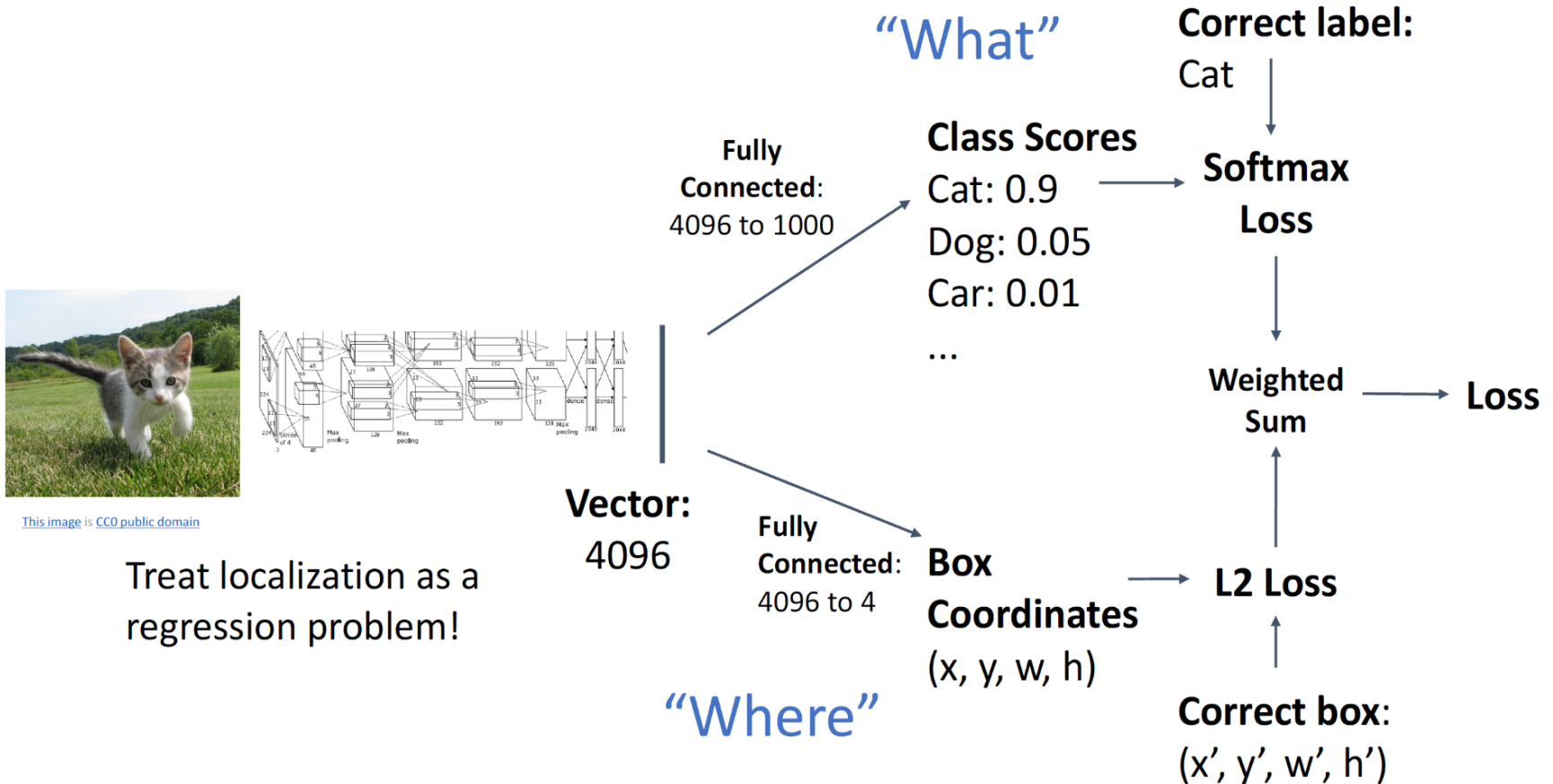


Confused with
Dissimilar Objects



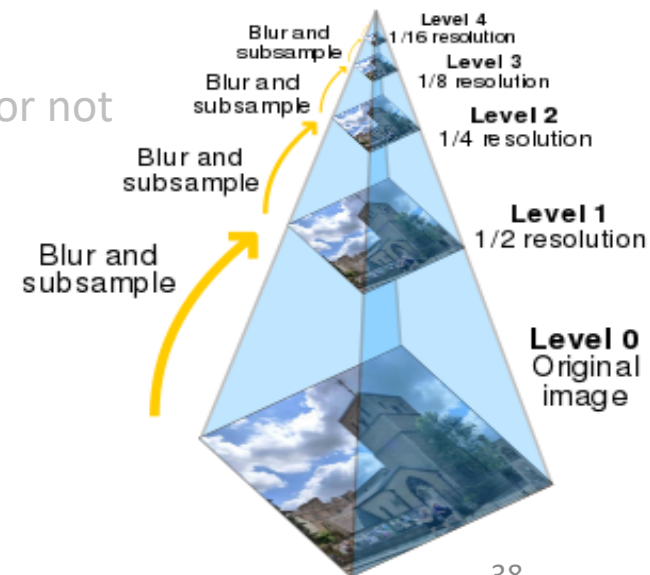
Detection by Classification

- CNN-based Methods



Detection by Classification (cont'd)

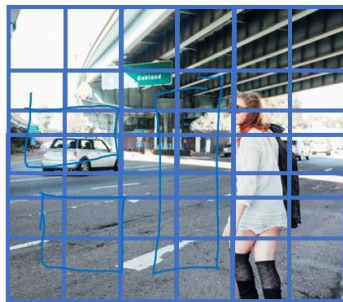
- Sliding Windows
 - “Slide” a box within the input image, or even across image scales (see below)
 - Classify each cropped image region inside the box and determine if it’s an object of interest or not
 - E.g., HOG (person) detector by Dalal and Triggs (2005)
Deformable part-based model by Felzenswalb et al. (2010)
Real-time (face) detector by Viola and Jones (2001)
- Region (Object) Proposals
 - Generate region (object) proposals
 - Classify each proposal and determine if it’s an object or not



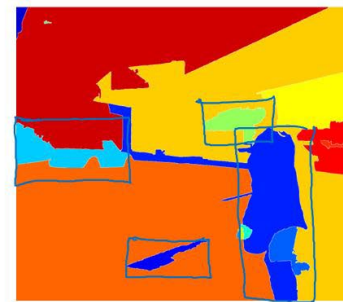
Detection by Classification (cont'd)

- Sliding Windows
 - “Slide” a box within the input image, or even across image scales (see below)
 - Classify each cropped image region inside the box and determine if it's an object of interest or not
- Region (Object) Proposals
 - Generate region (object) proposals -> how?
 - Classify each proposal and determine it's an object or not

Region proposal: R-CNN



↖



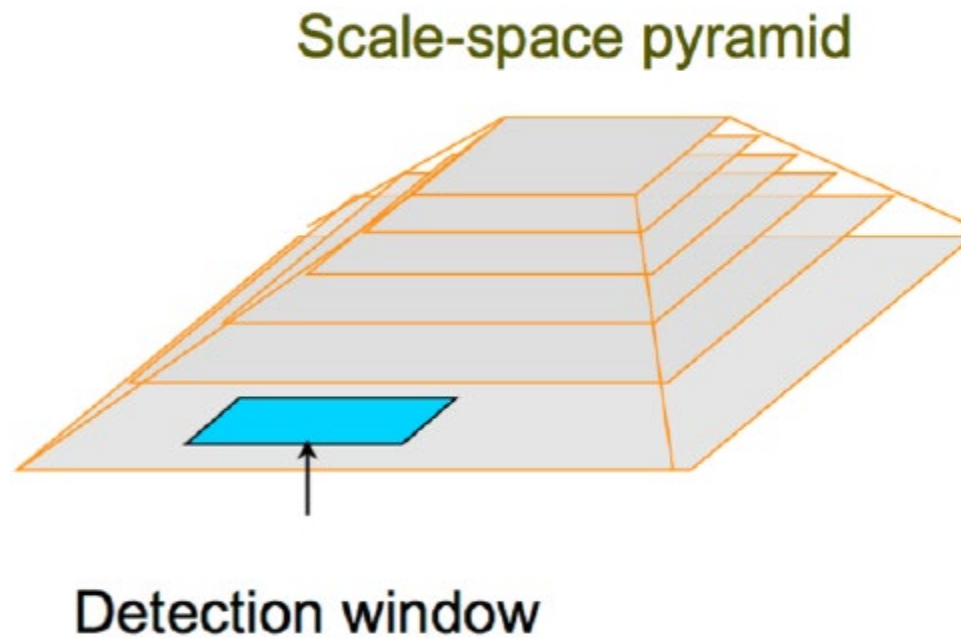
Segmentation algorithm

Before the Resurgence of CNN: The HOG Detector

- Histogram of Oriented Gradients (HOG)
- Sliding window detectors find the objects in 4 steps:
 - Inspect every window
 - Extract features in window
 - Classify & accept window if score > threshold
 - Clean-up (post-processing) stage



- Step 1: Inspect every window
 - Objects can vary in sizes, what to do?
 - Sliding window + image pyramid!



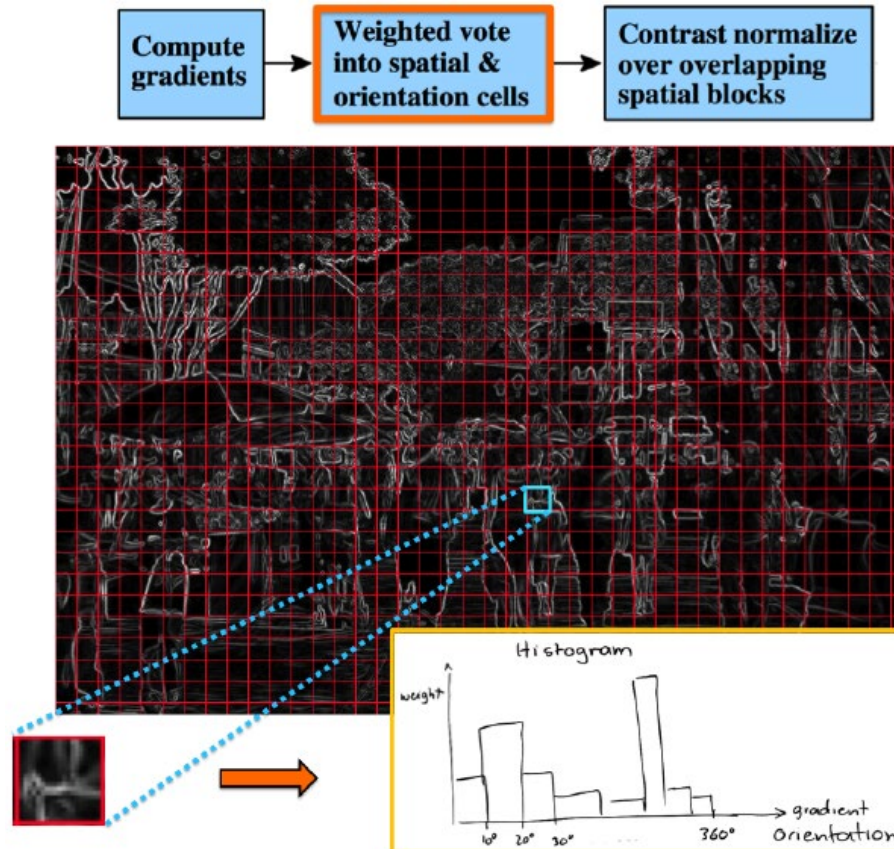
- Step 2: Extract Features in Window
 - Histogram of Gradients (HOG) features
 - Ways to compute image gradients...

Mask Type	1D centered	1D uncentered	1D cubic-corrected	2x2 diagonal	3x3 Sobel
Operator	$[-1, 0, 1]$	$[-1, 1]$	$[1, -8, 0, 8, -1]$	$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$
Miss rate at 10^{-4} FPPW	11%	12.5%	12%	12.5%	14%

(Miss rate: smaller is better)

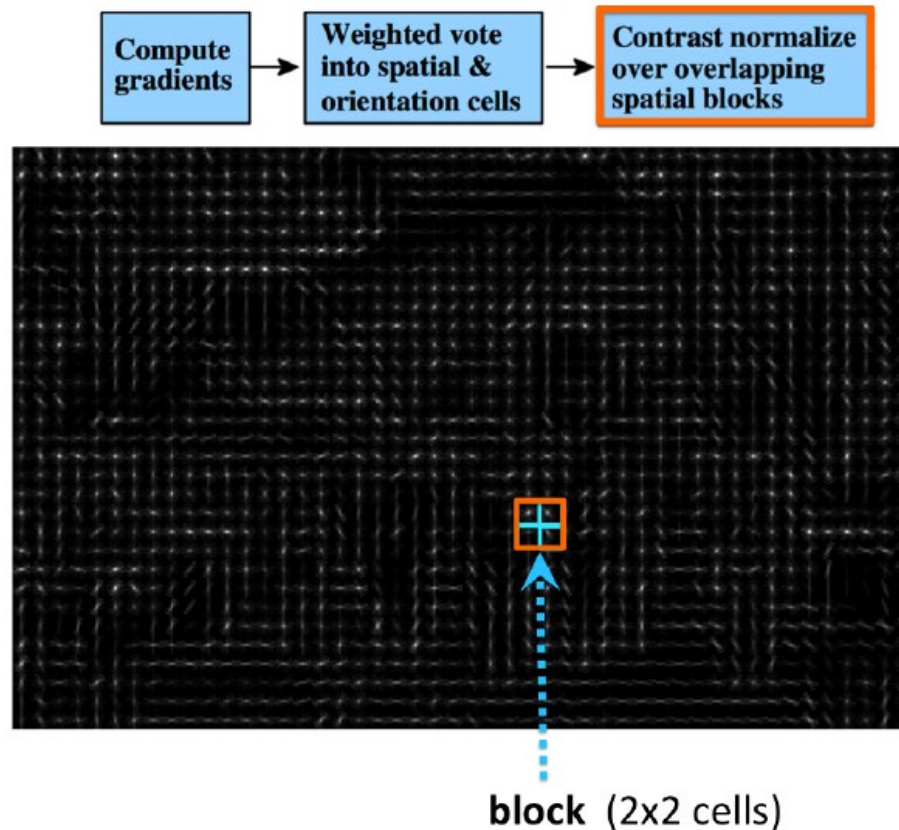
This gradient filter gives the best performance

- Step 2: Extract Features in Window
 - Histogram of Gradients (HOG) features
 - Divide the image into non-overlapping cells (grids) of 8 x 8 pixels
 - Compute a histogram of orientations in each cell, resulting in a 9-dimensional feature vector.



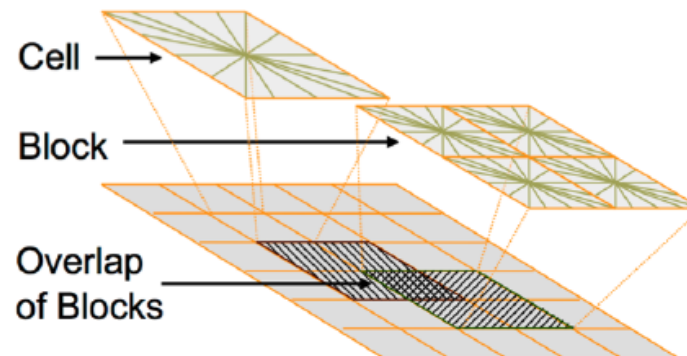
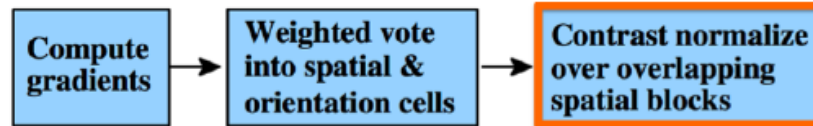
- Step 2: Extract Features in Window

- Histogram of Gradients (HOG) features
- Divide the image into non-overlapping cells (grids) of 8 x 8 pixels
- Compute a histogram of orientations in each cell (similar to SIFT), resulting in a 9-dimensional feature vector.
- We now take blocks, where each has 2 x 2 cells, for HOG normalization.



- Step 2: Extract Features in Window

- Compute a histogram of orientations in each cell (similar to SIFT), resulting in a 9-dimensional feature vector.
- We now take blocks, where each has 2 x 2 cells, for HOG normalization
- Normalize each feature vector, such that each block has unit norm. This does not change the dim of the feature, just the magnitude.



Feature vector $\mathbf{f} = [\dots, \dots, \dots]$

L2 normalization in each block:

$$\mathbf{f} = \frac{\mathbf{f}}{\sqrt{\|\mathbf{f}\|_2^2 + \epsilon^2}}$$

- Step 2: Extract Features in Window
 - Normalize each feature vector, such that each block has unit norm. This does not change the dim of the feature, just the magnitude.
 - For each class of *person*, window is 15 x 7 HOG cells.
 - Each cell is in 4 blocks thus has 4 different normalizations; we make each as a feature representation (in 3780 dimensions).
 - We vectorize each feature matrix in each window.



$$\# \text{ features} = 15 \times 7 \times 9 \times 4 = 3780$$

orientations

cells

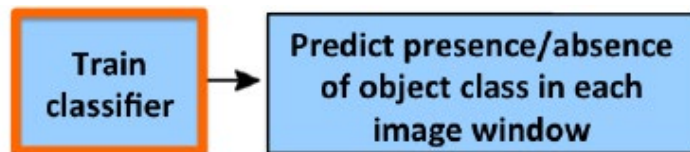
normalizations by neighboring cells

Final descriptor for window
(**person class in this case**)

- Step 3: Detection (classify & accept window if score > threshold)

- Training:

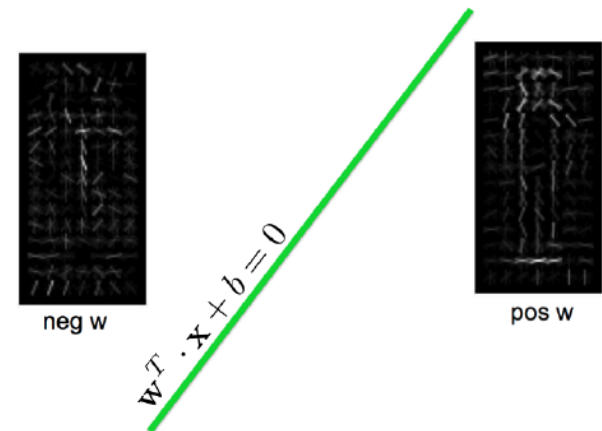
learn a linear/nonlinear classifier to predict the presence of object class in each window



positive training examples

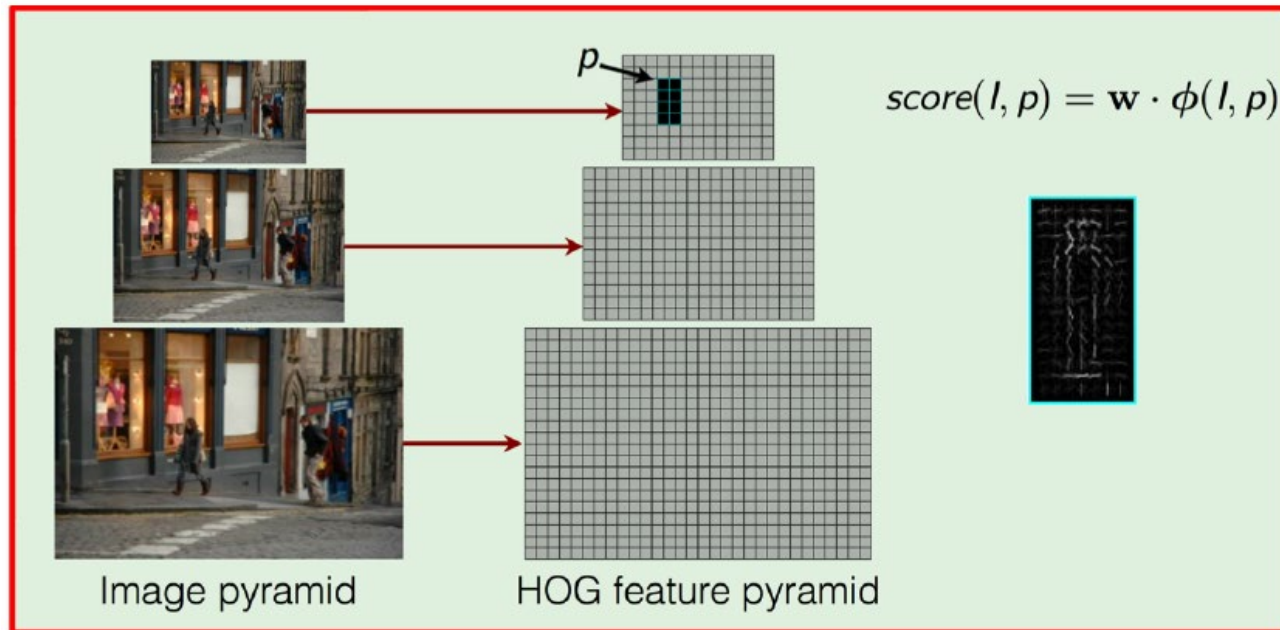


negative training examples



Train classifier. SVM (Support Vector Machines) is typically used.

- Step 3: Detection (Classify & accept window if score > threshold)
 - Training
 - Testing:
Compute the score $\mathbf{w}^T \mathbf{x} + \mathbf{b}$ in each location, which can be viewed as performing cross-correlation (or convolution) with template $\mathbf{w}$ (and add bias $\mathbf{b}$).



- Step 4: **Cleaning-Up**
 - Perform a greedy algorithm of **non-maxima suppression (NMS)** to pick the bounding box with highest score



Non-maxima suppression (NMS)

$$\text{overlap} = \frac{\text{area}(\text{box}_1 \cap \text{box}_2)}{\text{area}(\text{box}_1 \cup \text{box}_2)} > 0.5 \quad \Rightarrow \quad \begin{array}{|c|} \hline \text{remove} \\ \hline \text{box}_2 \\ \hline \end{array}$$

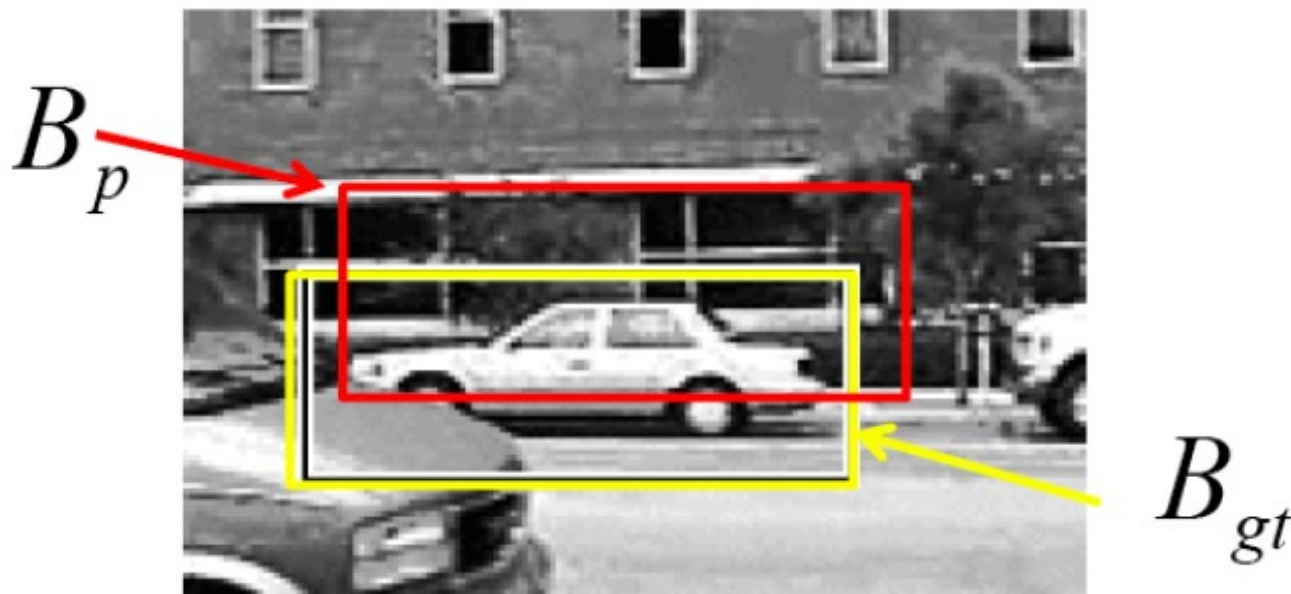
- Remove all boxes that overlap more than XX (typically 50%) with the chosen box

- **Evaluation**

- IoU (intersection over union)

- E.g, detection is correct if IoU between bounding box and ground truth > 50%

$$a_0 = \frac{\text{area}(B_p \cap B_{gt})}{\text{area}(B_p \cup B_{gt})}$$

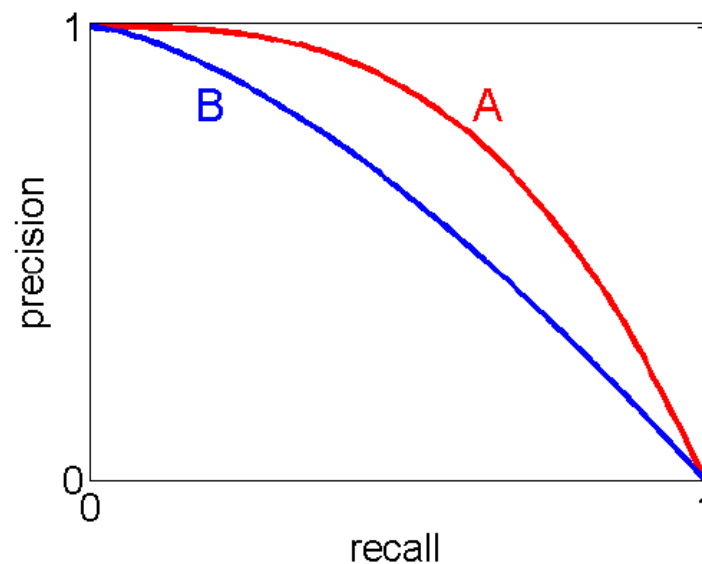


- **Evaluation**

- IOU (intersection over union)
 - Mean IOU (mIOU): average IOU across classes
- Precision & Recall
 - Sort all the predicted boxes according to scores, in a descending order
 - For each location in the sorted list, we compute precision and recall obtained when using top k boxes in the list.

$$\text{recall} = \frac{\# \text{correct boxes}}{\# \text{ground-truth boxes}}$$

$$\text{precision} = \frac{\# \text{correct boxes}}{\# \text{all predicted boxes}}$$



- Evaluation

- IOU (intersection over union)

- Precision and Recall

$$\text{Precision} = \frac{\text{True Positive}}{\text{Predicted Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

$$\text{Recall} = \frac{\text{True Positive}}{\text{Actual Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

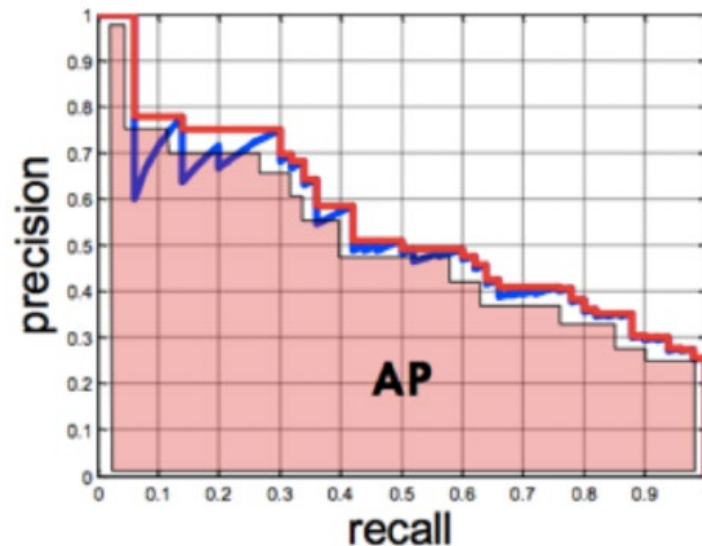
$$\text{Accuracy} = \frac{\text{True Positive} + \text{True Negative}}{\text{Total}}$$

		Actual (True) Values	
		Positive	Negative
Predicted Values	Positive	TP	FP
	Negative	FN	TN

- F1-score: harmonic mean of P & R, i.e.,

$$2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Evaluation (cont'd)
 - IoU (intersection over union)
 - Precision, Recall, F1-score
 - Average Precision (AP):
 - Compute the area under P-R curve
 - mean Average Precision (mAP): average of AP across classes



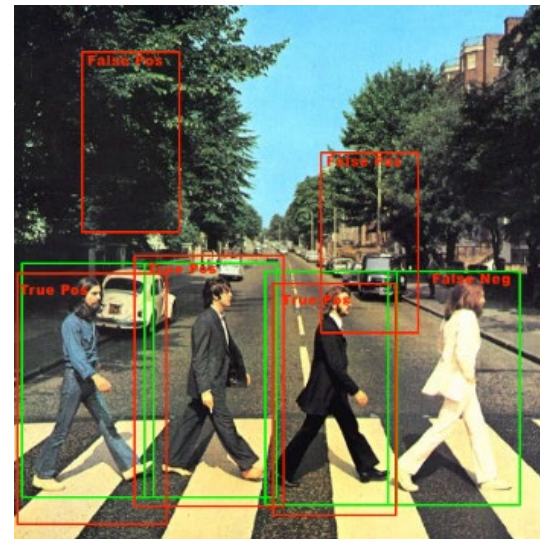
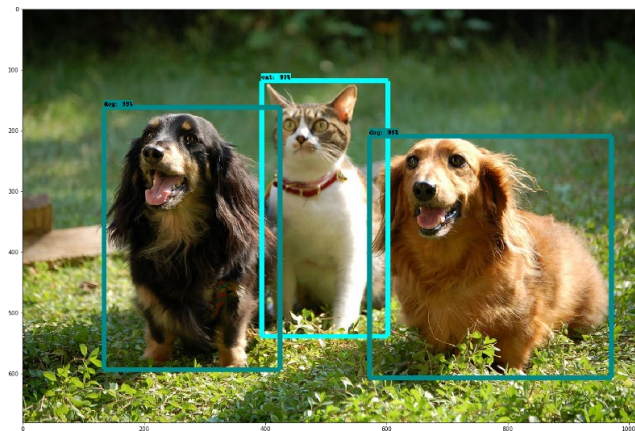
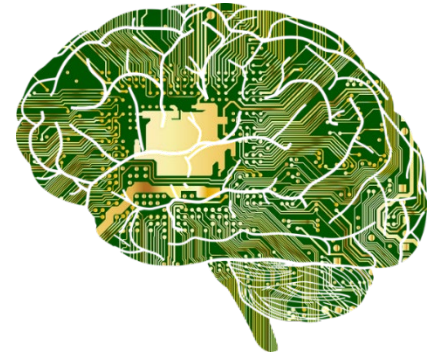
Something to Think About...

- Sliding window detectors work
 - *very well* for faces
 - *fairly well* for cars and pedestrians
 - *badly* for cats and dogs
- Why are some classes easier than others?

What to Covered Today...

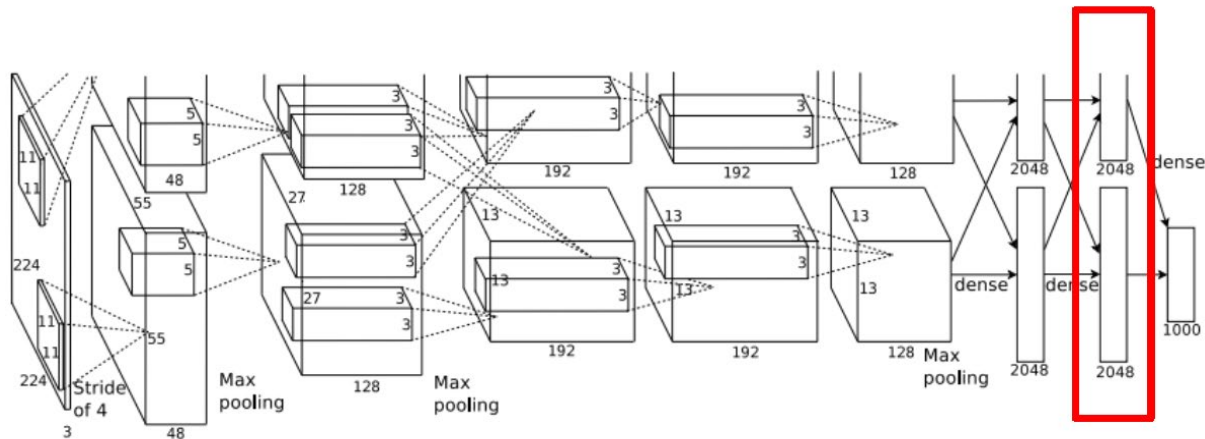
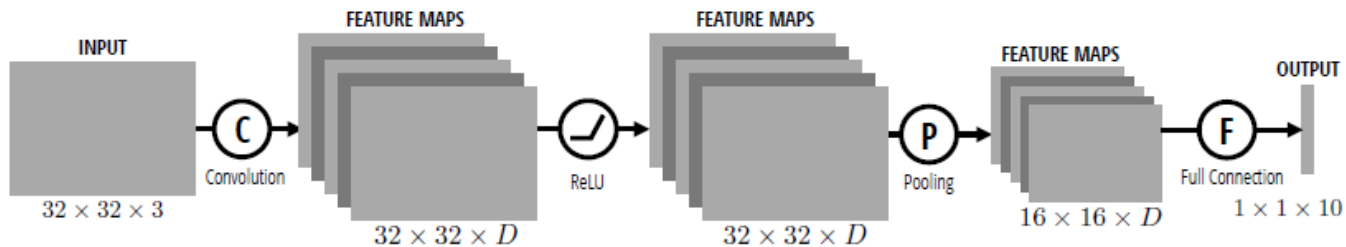
- **Object Detection**

- Detection via Sliding Windows
- Two-Stage vs. Single-Stage Detectors
- Transformer-based Detectors
- 3D Detection

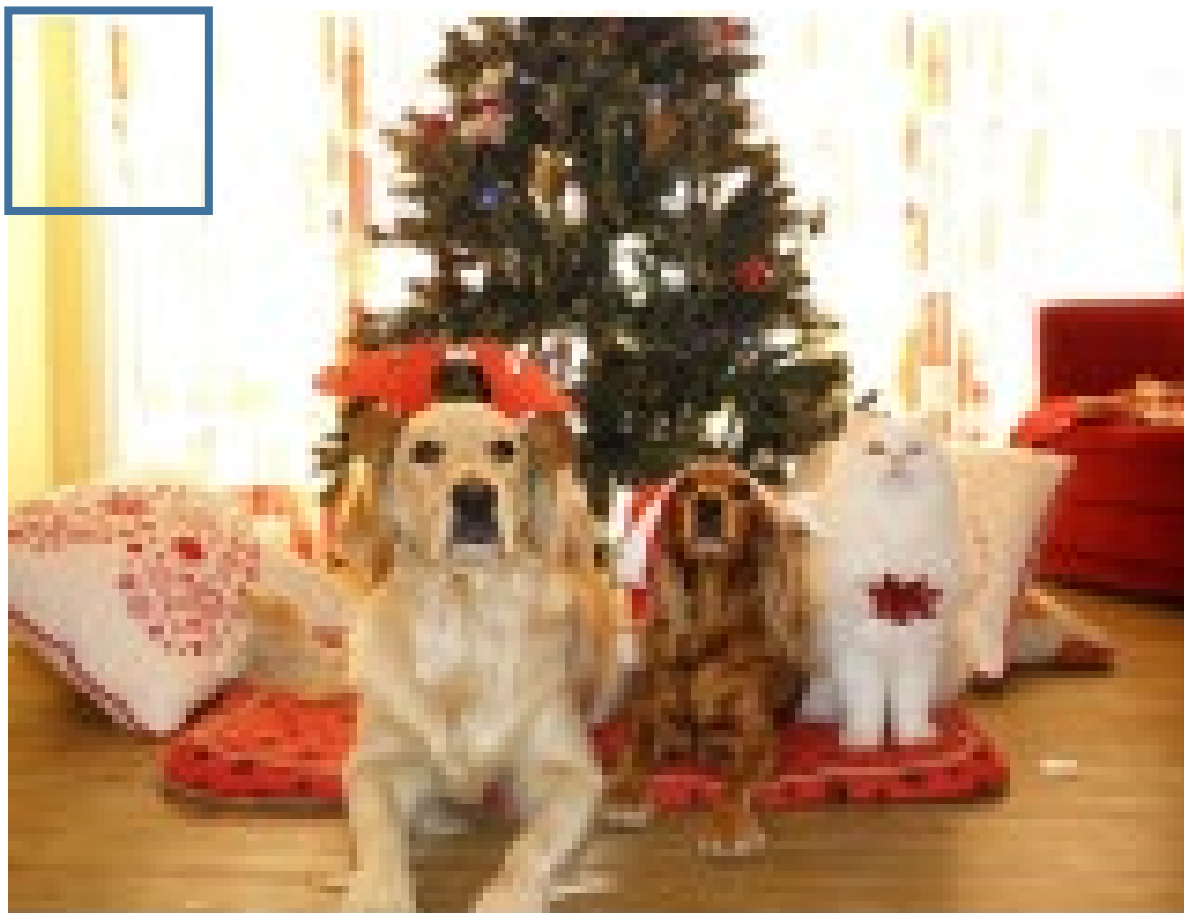


Recall that

- Visual Features derived by Convolutional Neural Networks

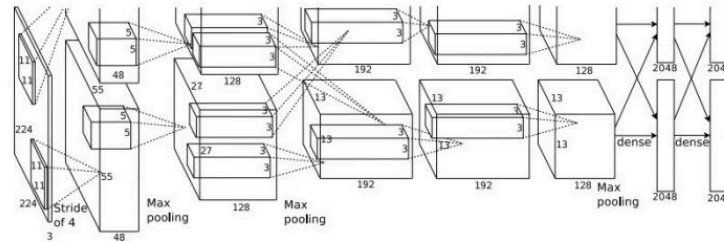


CNN as Feature Extractor



CNN as Feature Extractor

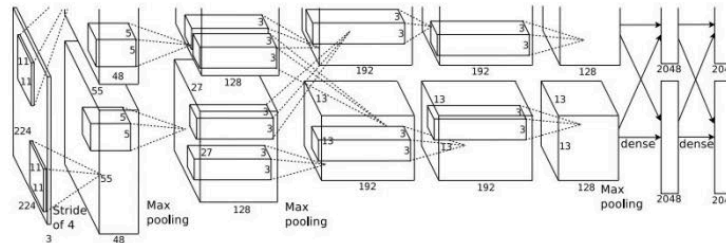
Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? YES
Cat? NO
Background? NO

CNN as Feature Extractor

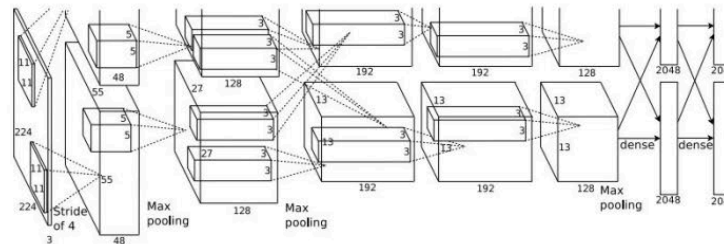
Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? YES
Cat? NO
Background? NO

CNN as Feature Extractor

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



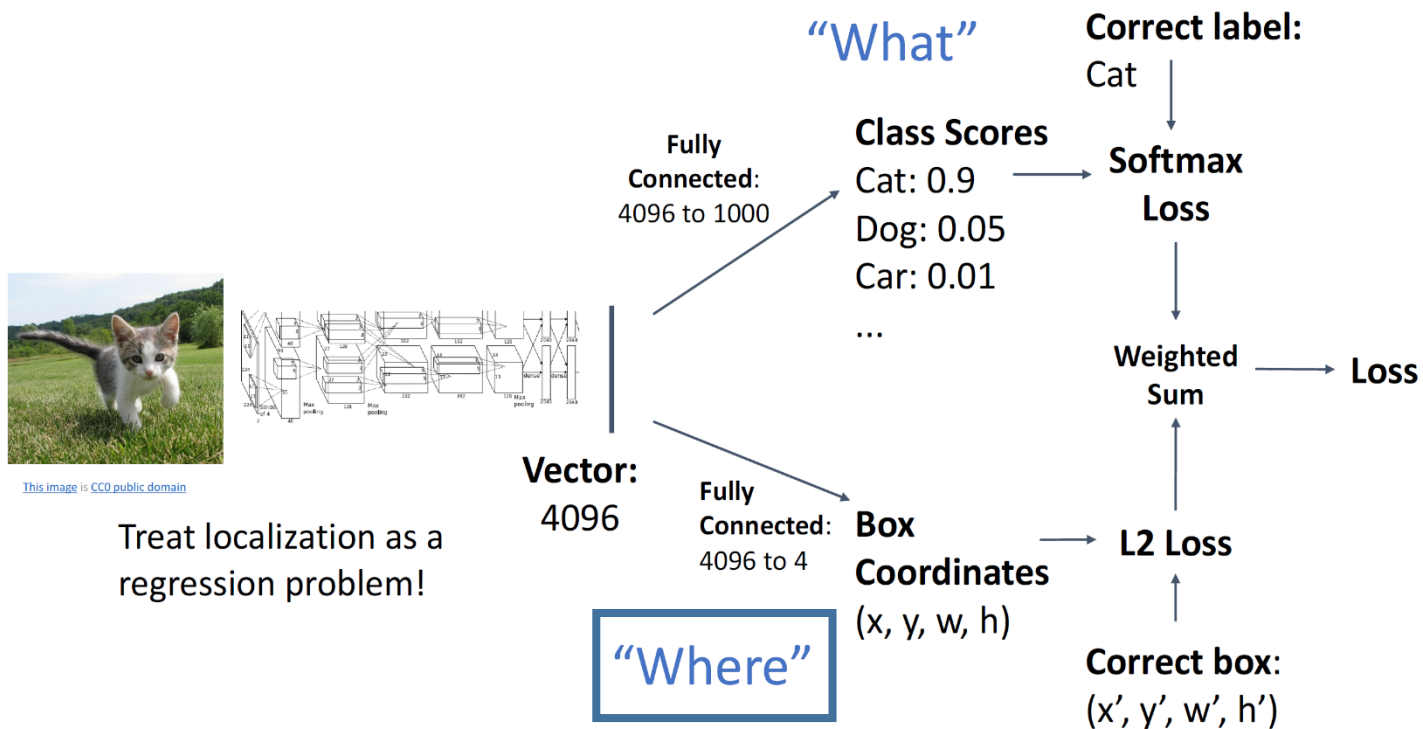
Dog? NO
Cat? YES
Background? NO

CNN as Feature Extractor

- What could be the problems?
 - Suppose we have an image of 600 x 600 pixels.
If sliding window size is 20 x 20,
then have $(600-20+1) \times (600-20+1) = \sim 330,000$ windows to compute.
 - What if more accurate results are needed,
need to perform **multi-scale detection** by
 - Resize image
 - Multi-scale/shape sliding windows
 - For each image, we need to forward pass image regions through CNN
for at least $\sim 330,000$ times. -> **Slow!!!**

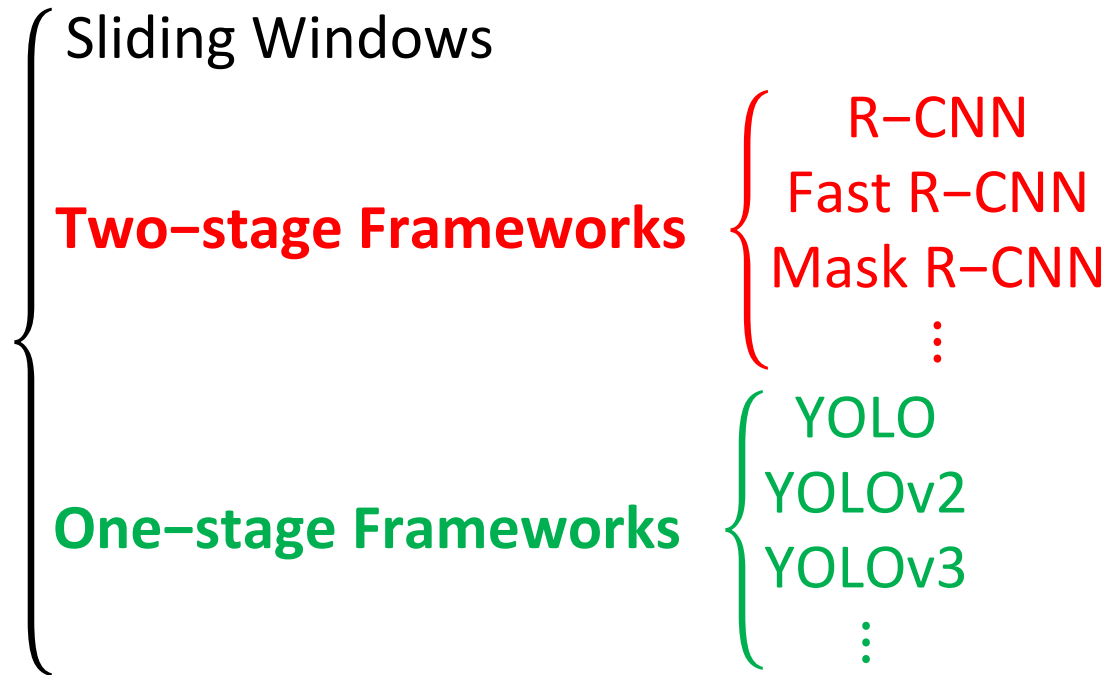
Recap: CNN for Object Detection

- Need to deal with more than one object
 - How?



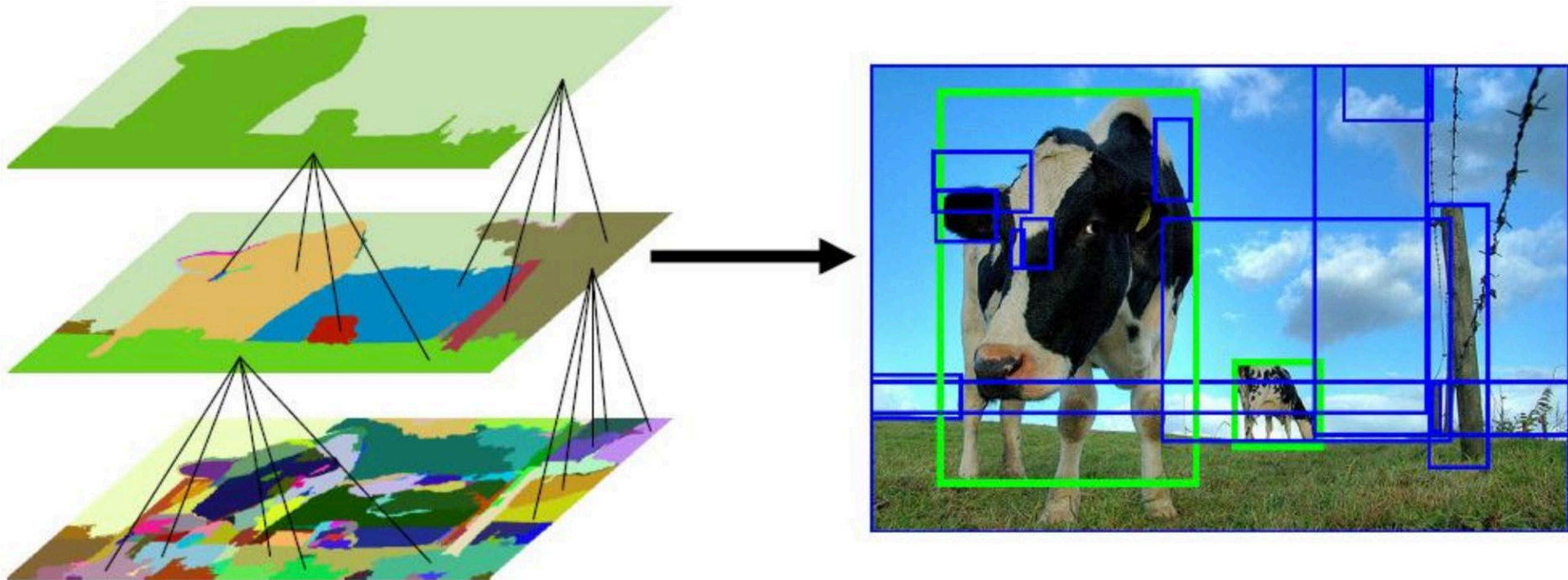
Two-Stage vs. One-Stage Object Detection

Methods

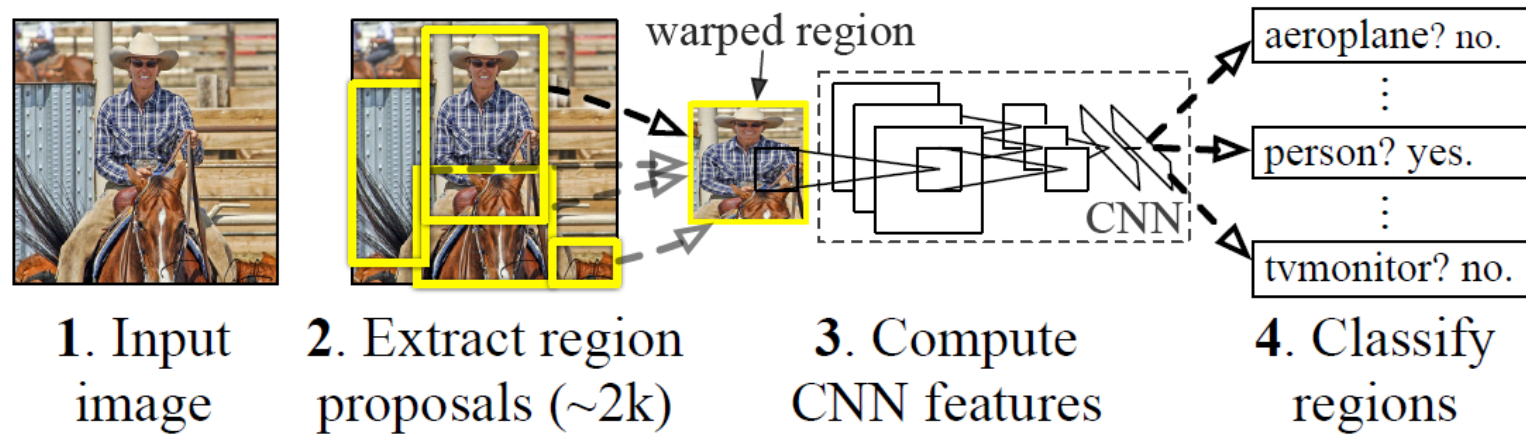


Region Proposal

- Solution
 - Use pre-processing algorithms to filter out some regions first, and feed the regions of interest (i.e., region proposals) into CNN
 - E.g., selective search

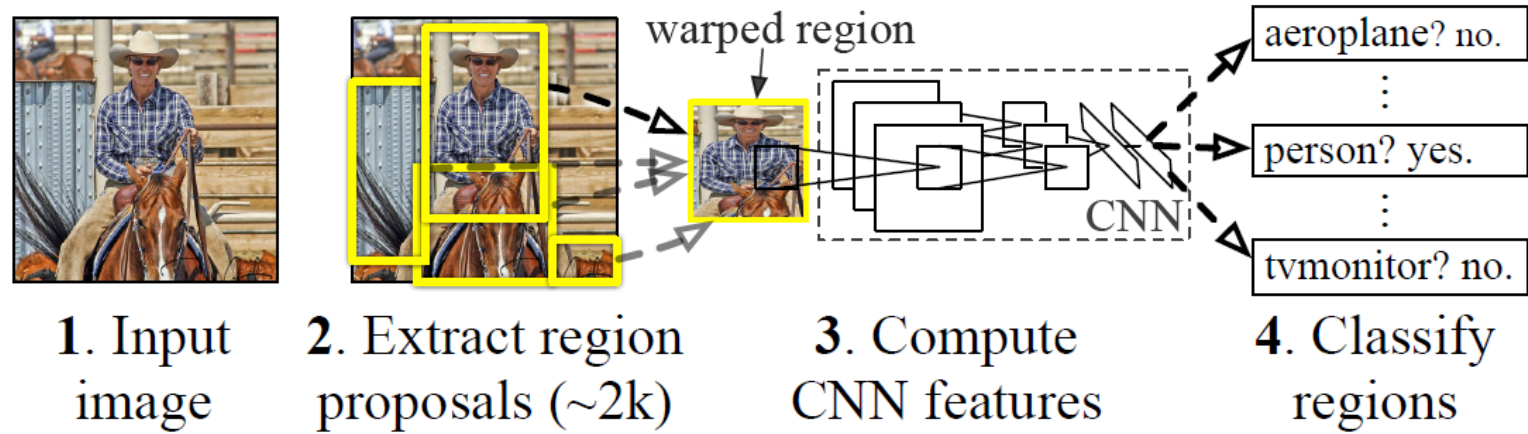


R-CNN (Girshick et al. CVPR 2014)



- Replace sliding windows with “selective search” region proposals (Uijlings et al. IJCV 2013)
- Extract rectangles around regions and resize to **227x227** pixels
- Extract features with fine-tuned CNN (e.g., initialized with network pre-trained on ImageNet)
- Classify last layer of network features with linear classifiers (e.g., SVM/MLP), and refine bounding box localization (bbox regression) simultaneously

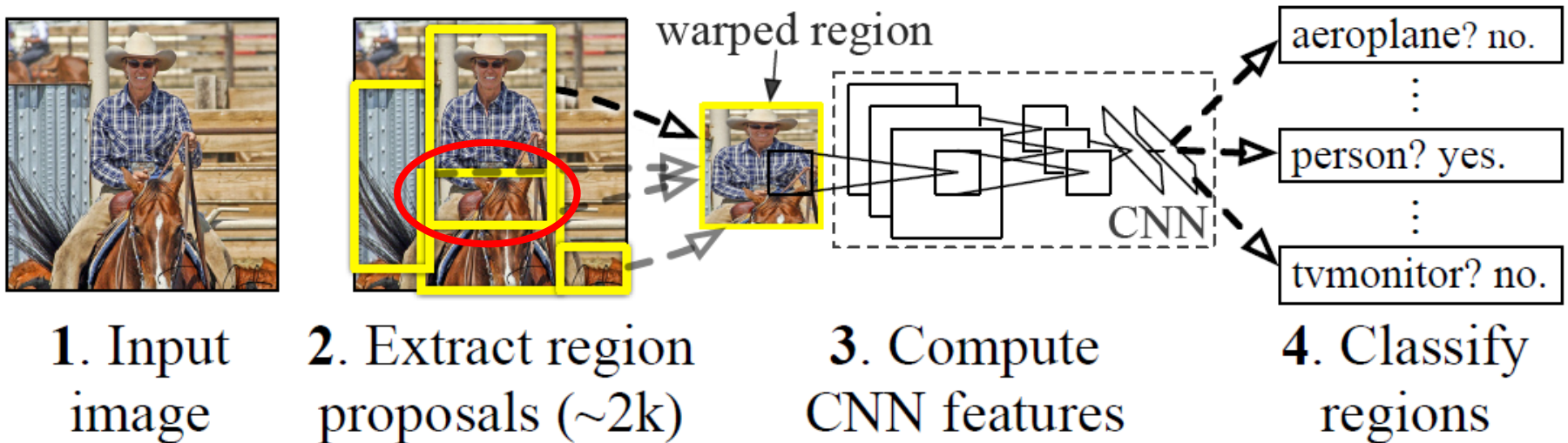
R-CNN (Girshick et al. CVPR 2014)



- Ad hoc training objectives:
 - Object class: Fine-tune network with softmax classifier (log loss)
 - Object class: Train post-hoc linear SVMs for each class (hinge loss)
 - Bbox location: Train post-hoc bounding-box regressors (least squares loss)
- Training is extremely slow with lots of disk space.
- Implementation/testing cannot be done in real time.

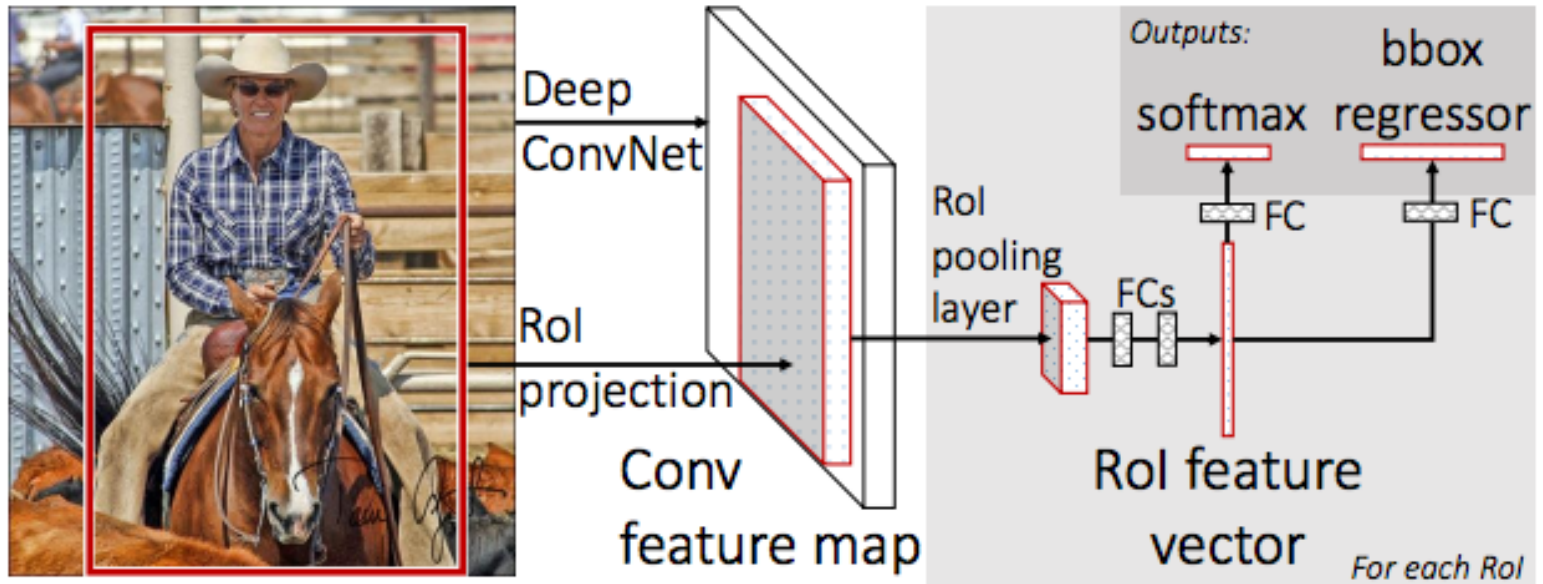
R-CNN (Girshick et al. CVPR 2014)

- What could be the problems?
 - **Repetitive computation!**
For overlapping regions, we feed it multiple times into CNN



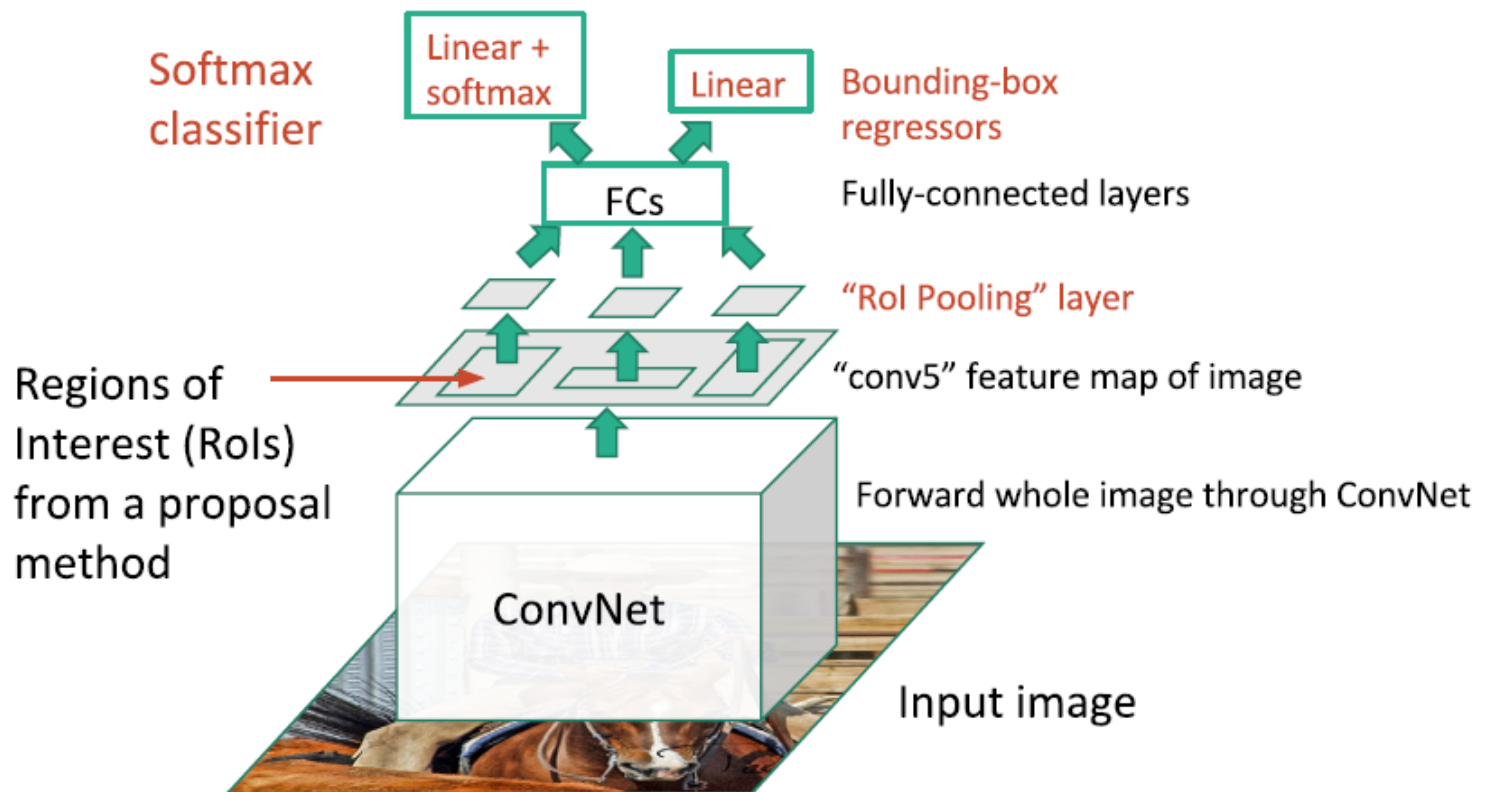
Fast R-CNN (Girshick ICCV 2015)

- Solution
 - Why not feed the whole image into CNN **only once**?
 - Then, crop the feature map instead of the image itself



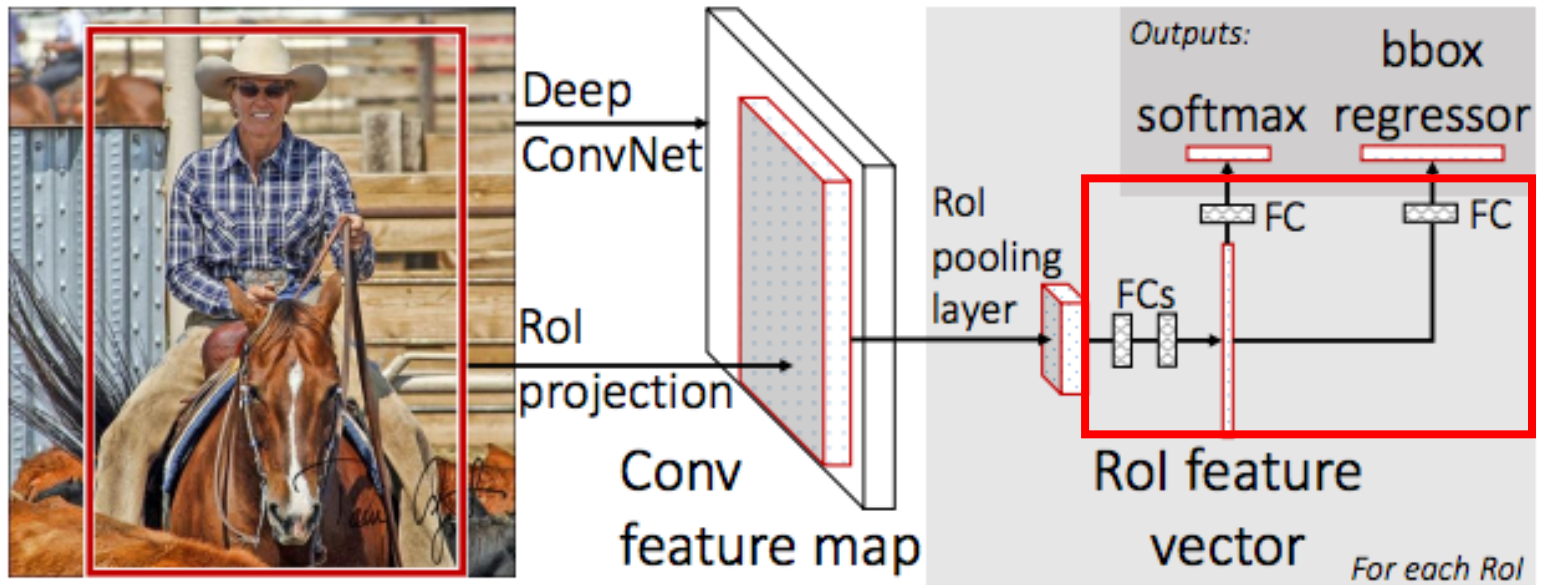
Fast R-CNN (Girshick ICCV 2015)

- Solution



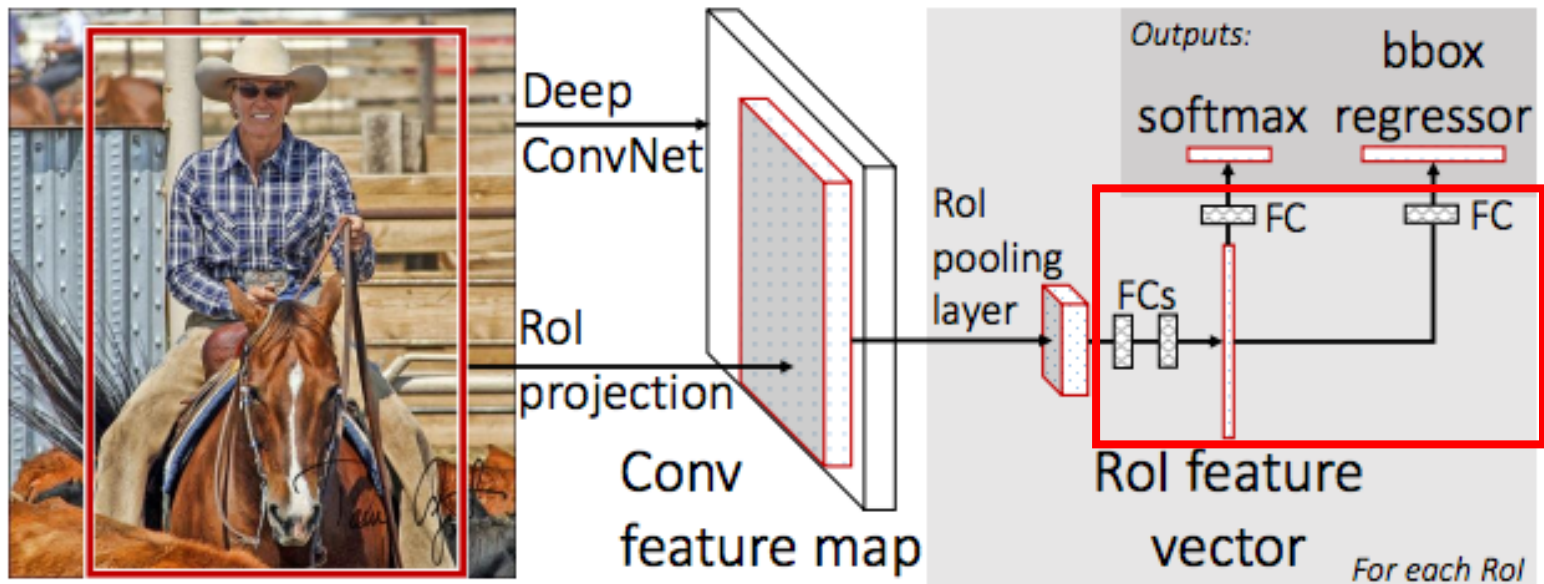
Fast R-CNN (Girshick ICCV 2015)

- Wait...what about the image feature to be processed?
 - Any potential problem?



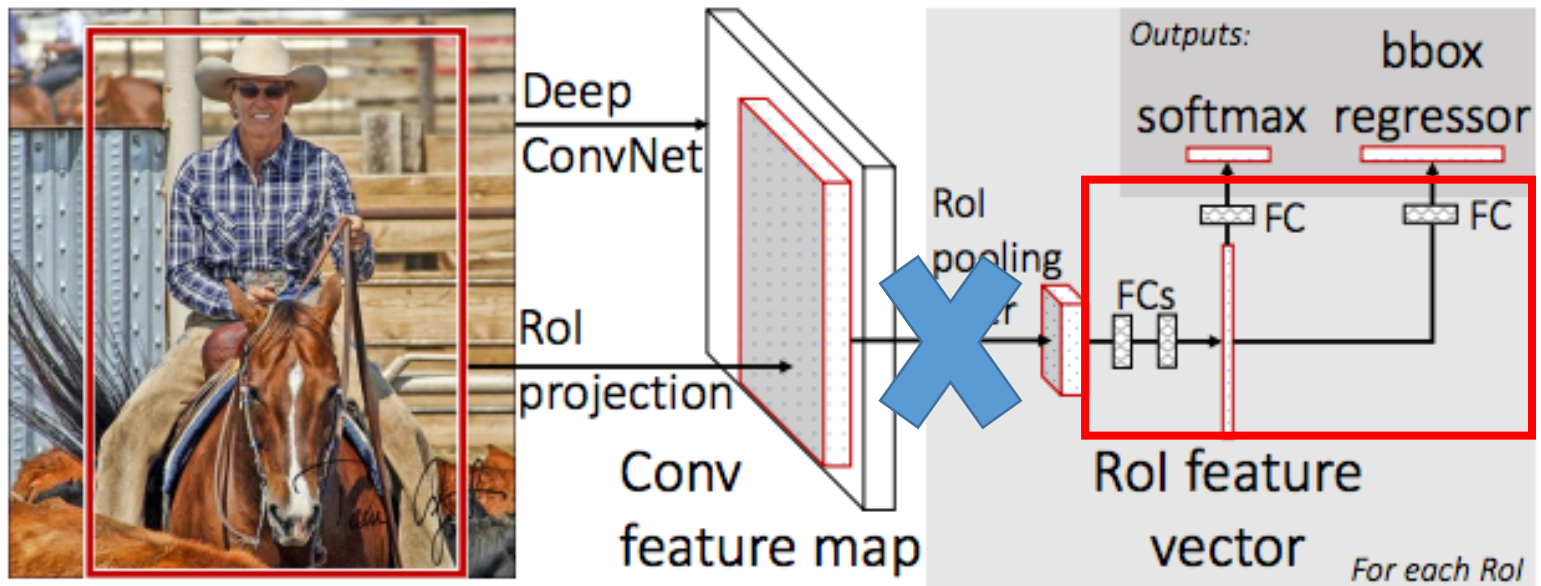
Fast R-CNN (Girshick ICCV 2015)

- Features need to be of the same size.
- How to crop features?
 - Since we have fully-connected layers, the size of feature map for each bounding box should be a **fixed number**



Fast R-CNN (Girshick ICCV 2015)

- How to crop features?
 - Since we have fully-connected layers, the size of feature map for each bounding box should be a fixed number
 - Resize/Interpolate the feature map as fixed size?
 - Not optimal. This operation is hard to backprop.
 - > we cannot train the conv layers in CNNs...

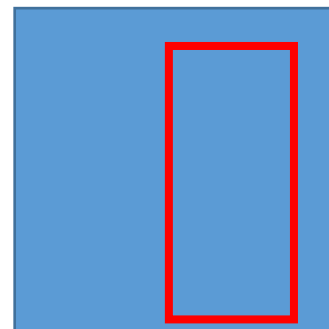


Fast R-CNN (Girshick ICCV 2015)

- How to crop features?
 - Since we have fully-connected layers, the size of feature map for each bounding box should be a fixed number
 - Resize/Interpolate the feature map as fixed size?
 - Not optimal. This operation is hard to backprop.
-> we cannot train the conv layers for this problem...
- RoI Pooling
 - How to crop regions of various sizes but resulting in the same size?

Rol Pooling

- Step 1:
Get bounding box for feature map from bounding box for image
 - Due to the (down)convolution/pooling operations, feature map would have a smaller size than the original image.



Feature map

Rol Pooling

- Step 2:
Divide cropped feature map into fixed number of sub-regions
 - The last column and last row might be smaller

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Feature map
4 x 4 x 1

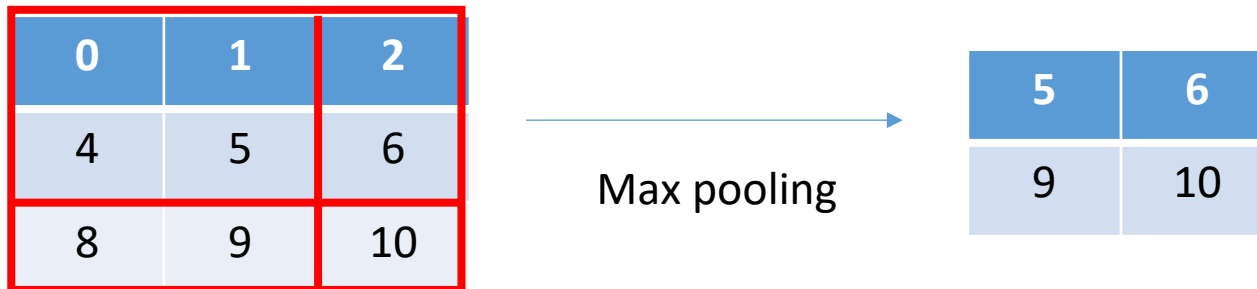
Make it as
2x2 grids

0	1	2
4	5	6
8	9	10

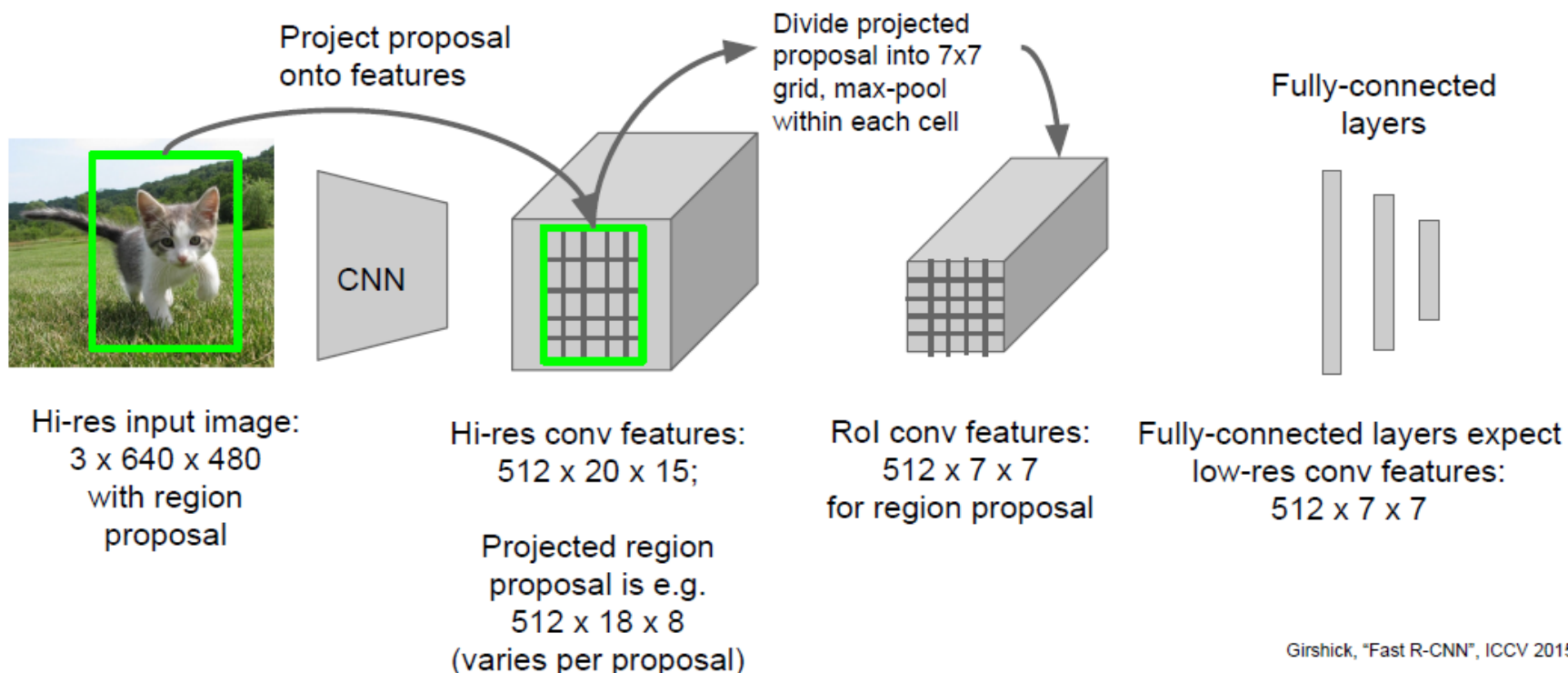
1	2	3
5	6	7
9	10	11
13	14	15

Roi Pooling

- Step 3:
For each sub-region, perform max pooling (pick the max one)

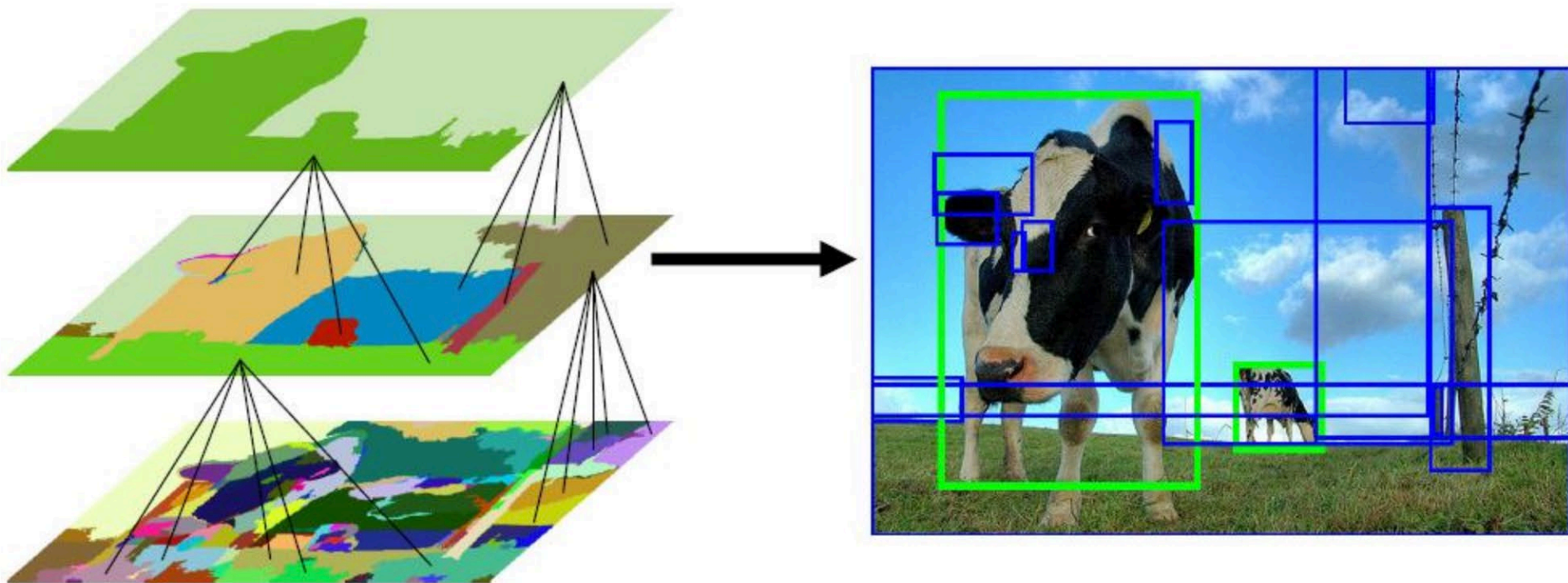


Rol Pooling



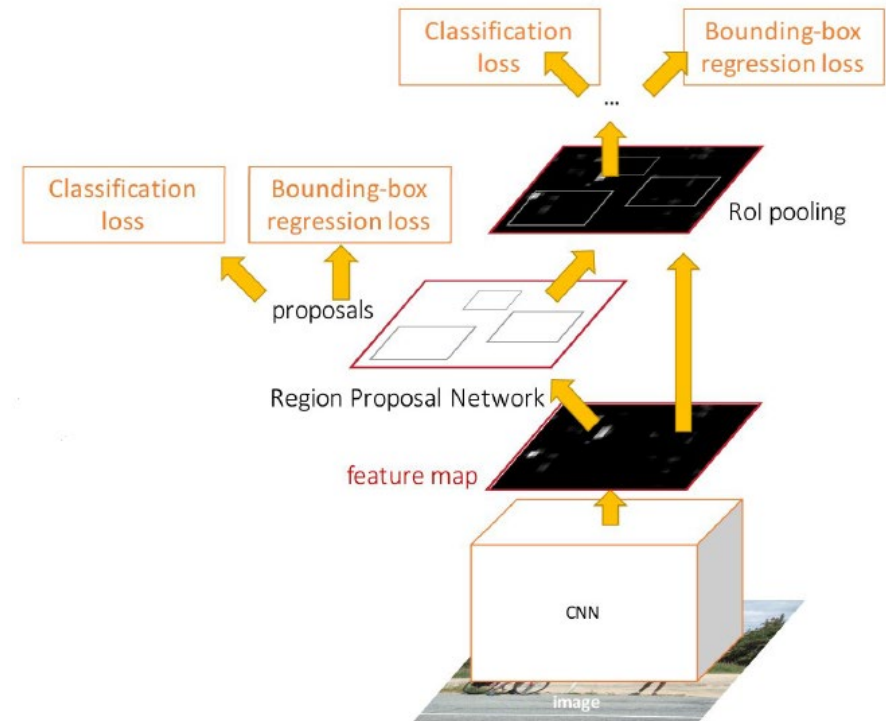
Fast R-CNN (Girshick ICCV 2015)

- What could be the problems?
 - We still need to collect the region proposals from a **pre-processing step**, which does not allow **end-to-end** learning.



Faster R-CNN (Ren et al. NIPS/NeurIPS 2015)

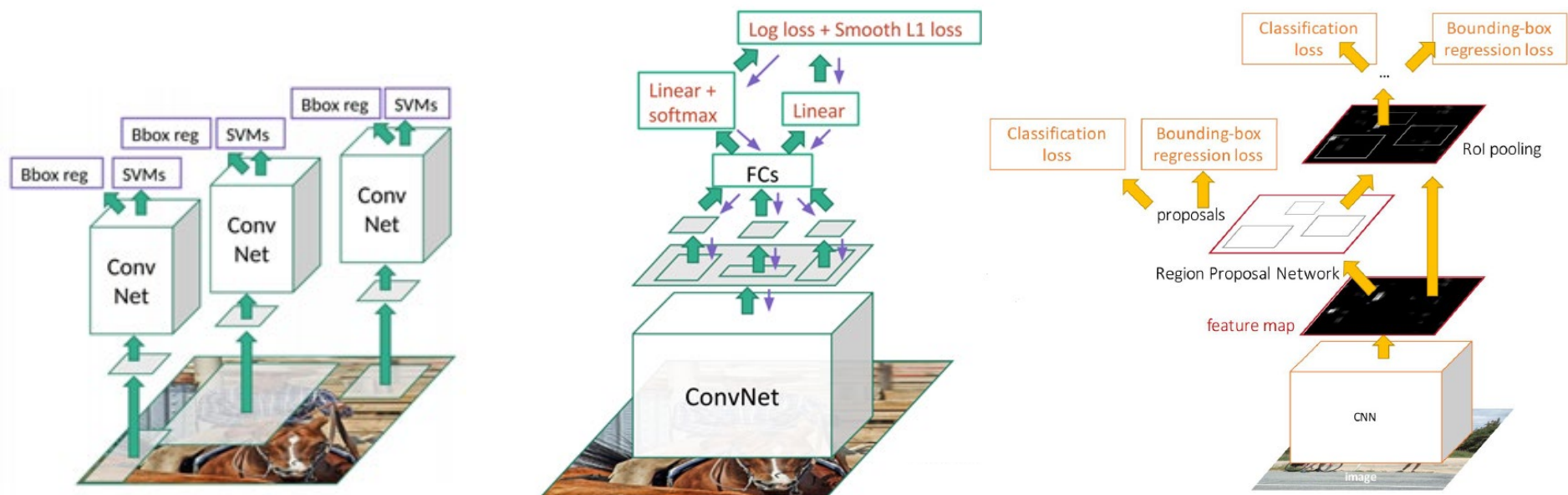
- Solution
 - Why not generate region proposals using CNN?
 - > Insert **Region Proposal Network (RPN)** to predict proposals from features
 - Jointly train with 4 losses:
 - RPN classification loss
 - RPN regress box coordinates
 - Final classification loss
 - Final box coordinates



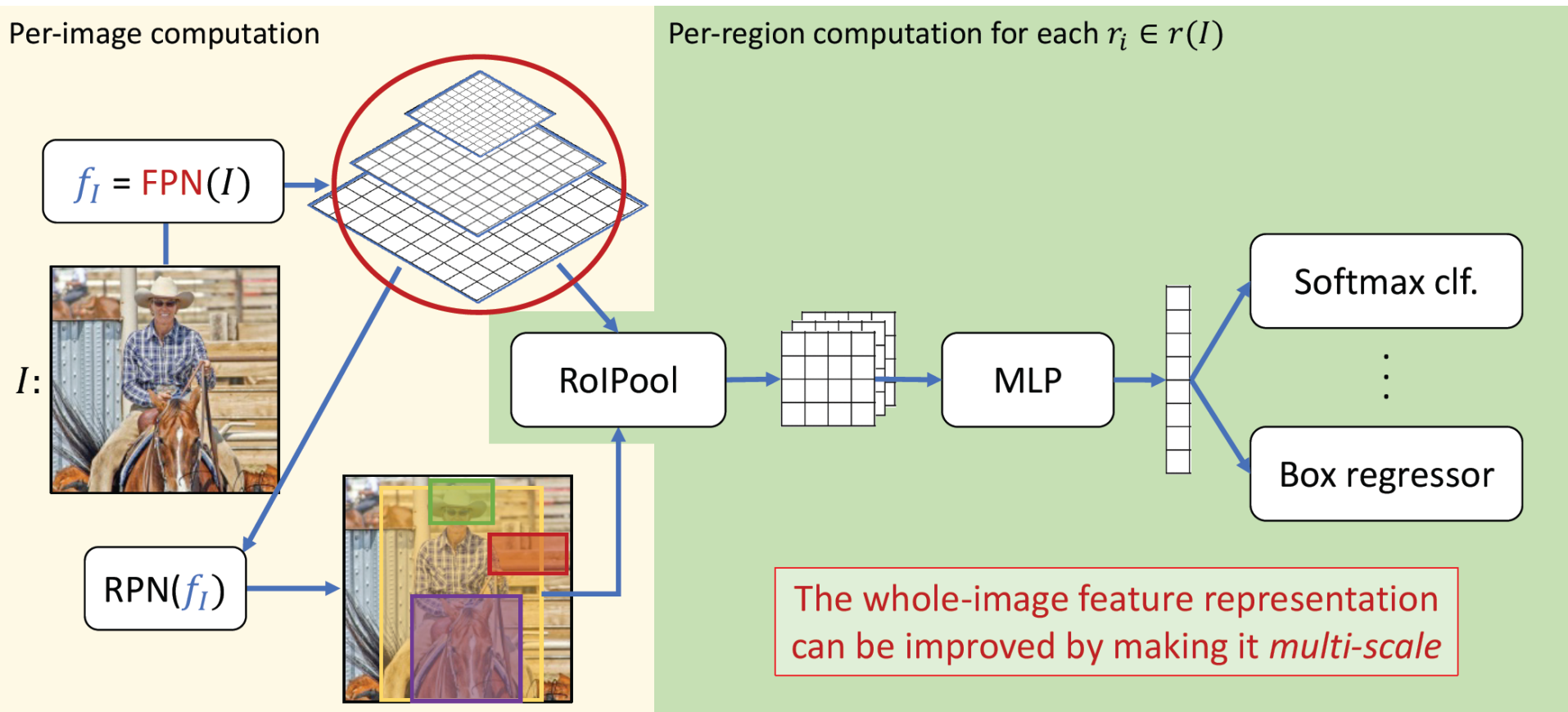
<https://arxiv.org/pdf/1506.01497.pdf>

Image credit: http://zh.gluon.ai/chapter_computer-vision/object-detection.html

R-CNN, Fast R-CNN, & Faster R-CNN



Faster R-CNN with Feature Pyramid Network

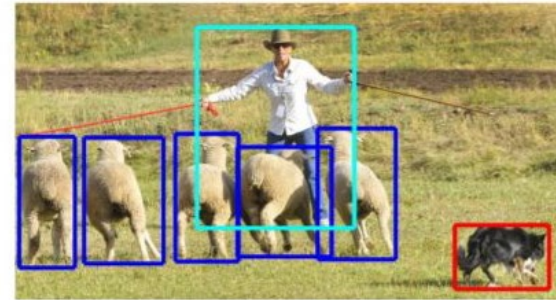


Faster R-CNN (Ren et al. NIPS 2015)

- What could be the problems
 - **Two-stage detection** pipeline is still too slow for real-time detection in videos...
 - What about **instance-wise information**?



(a) Image classification



(b) Object localization



(c) Semantic segmentation

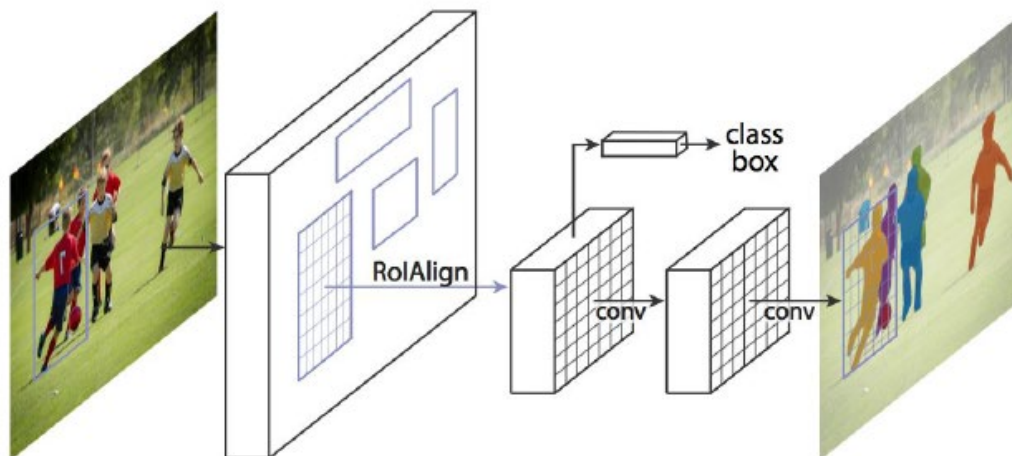
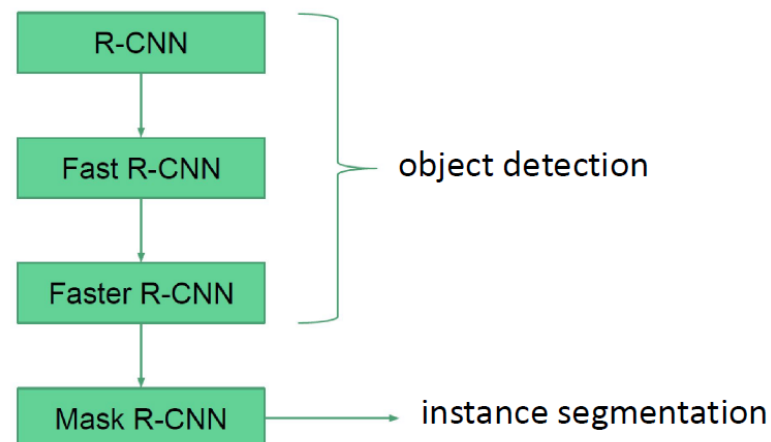


(d) **Instance segmentation**

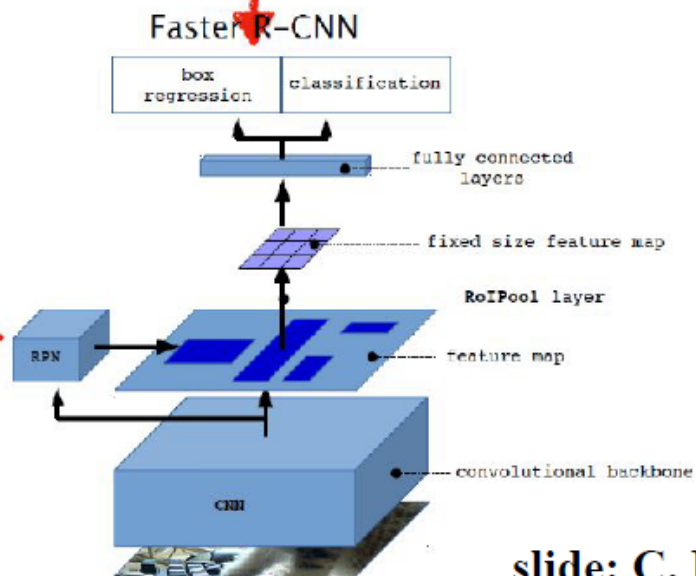
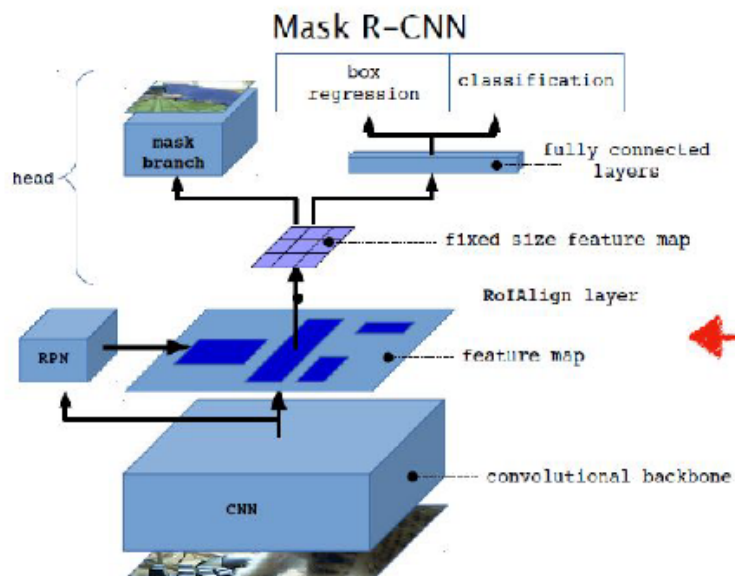
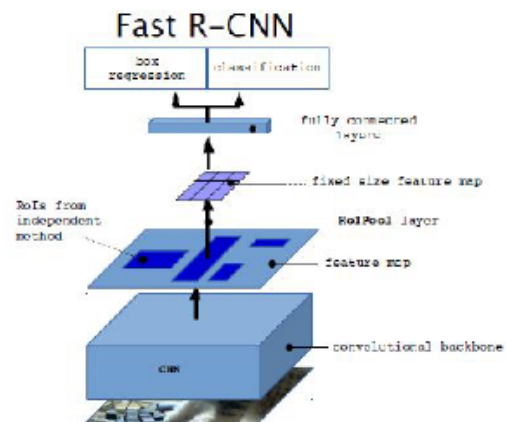
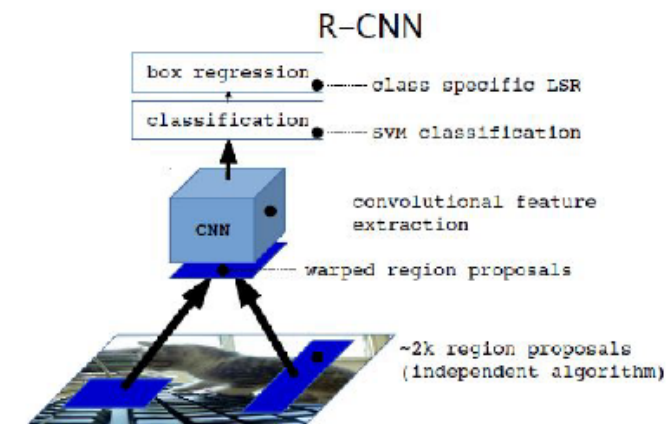
Mask R-CNN (ICCV2017)

- Still a 2-stage detector
- Goals:
 - Refined detection + precise segmentation
 - Faster R-CNN + FCN

- Overall design:
 - Use of ResNet or feature pyramid net for feature extraction
 - Use RPN to produce proposals (w/ ROI align)
 - Use one detection branch for box classification + regression
 - Use one



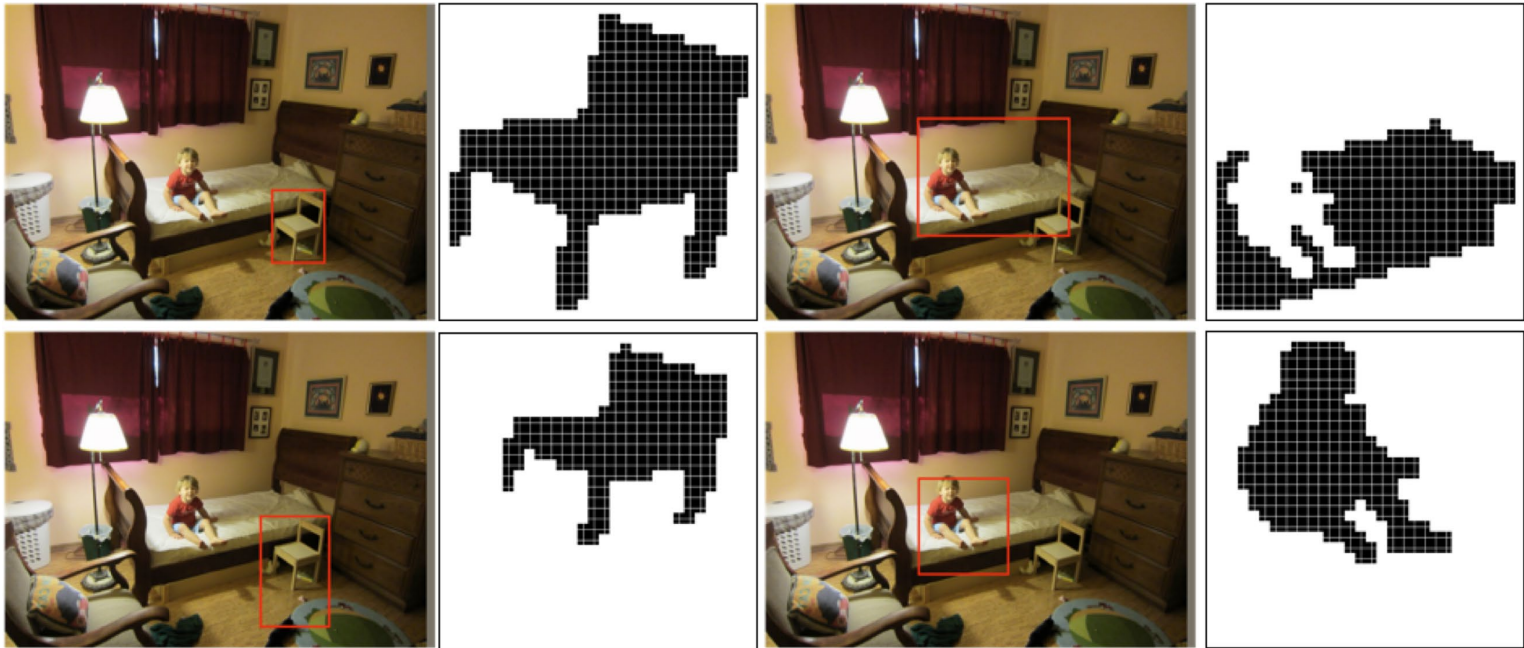
R-CNN Family



slide: C. Lim

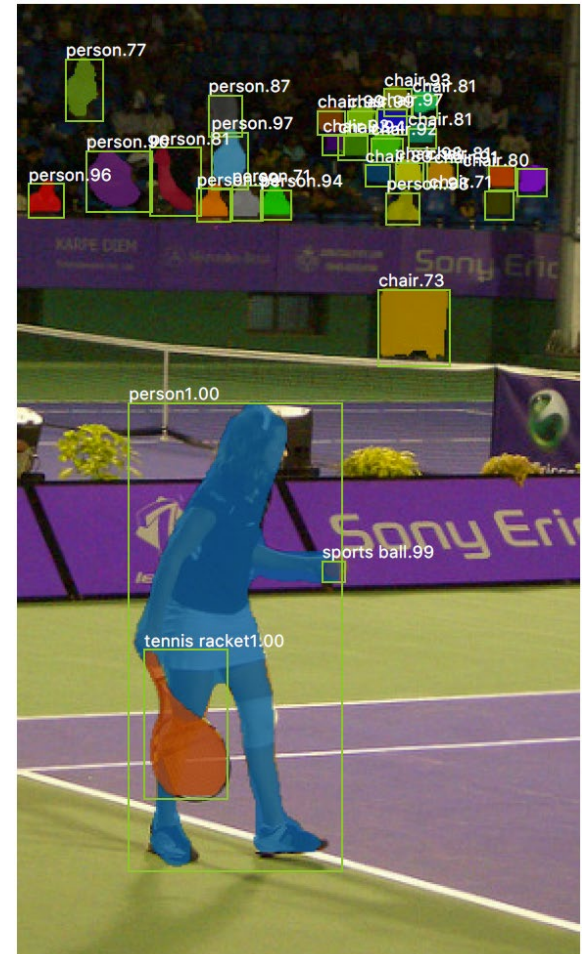
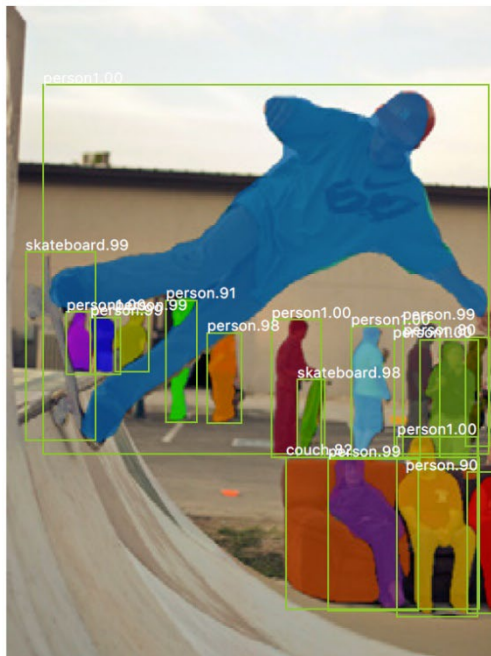
Mask R-CNN (cont'd)

- Example Training Data (requires pixel-level labels)



Mask R-CNN

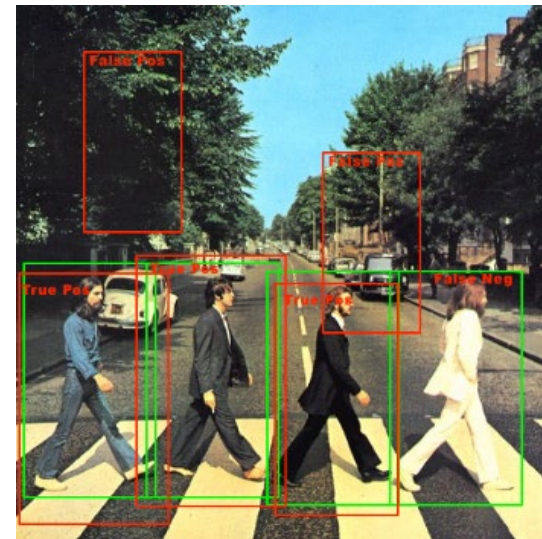
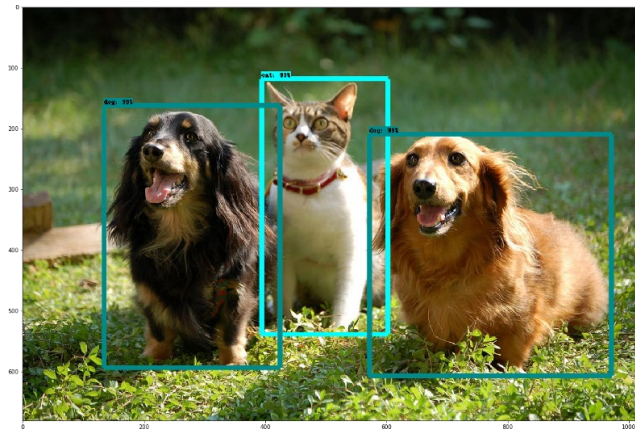
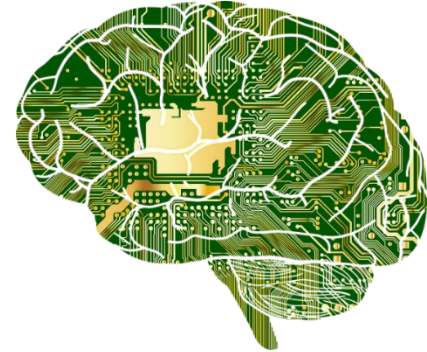
- Very good results!
 - running at 5fps though



What to Covered Today...

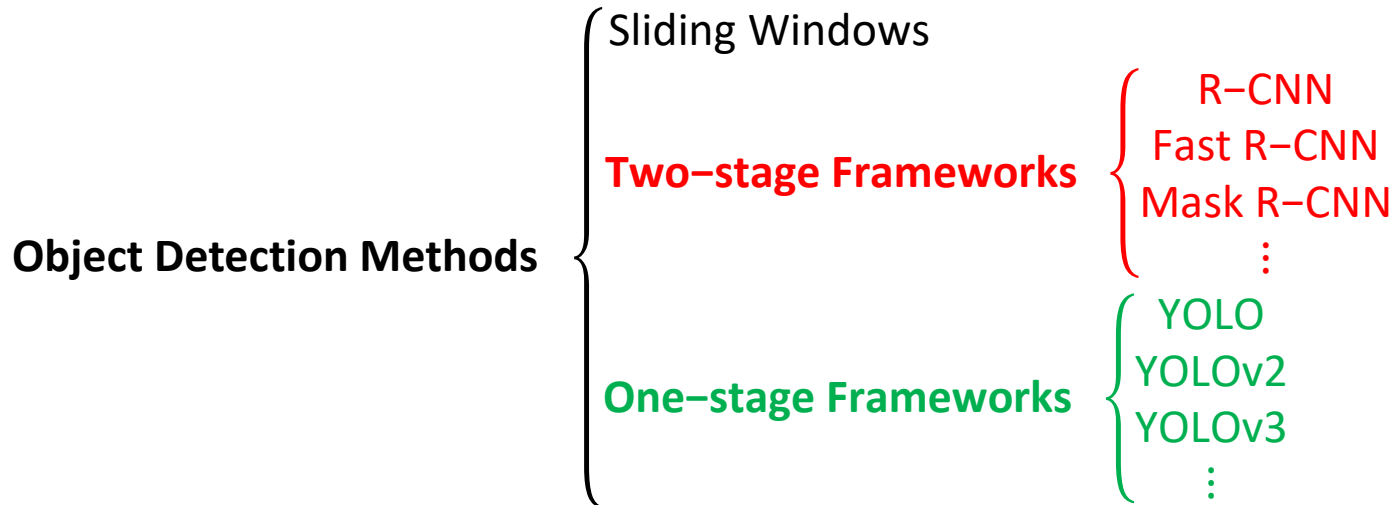
- **Object Detection**

- Detection via Sliding Windows
- Two-Stage vs. Single-Stage Detectors
- Transformer-based Detectors
- 3D Detection



Recap

- So far, the introduced methods follow a **two-stage** framework.
 1. Region Proposal
 2. Per-Region Classification/Regression
- Can we make it faster by integrating the above two steps into **one single network**?

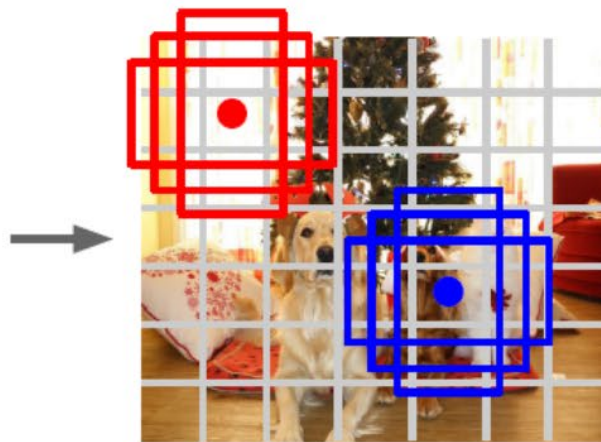


One-Stage Object Detection: Detection without Proposals

Go from input image to tensor of scores with one big convolutional network! →



Input image
 $3 \times H \times W$



Divide image into grid
 7×7
Image a set of **base boxes**
centered at each grid cell
Here $B = 3$

Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers:
($dx, dy, dh, dw, confidence$)
- Predict scores for each of C classes (including background as a class)

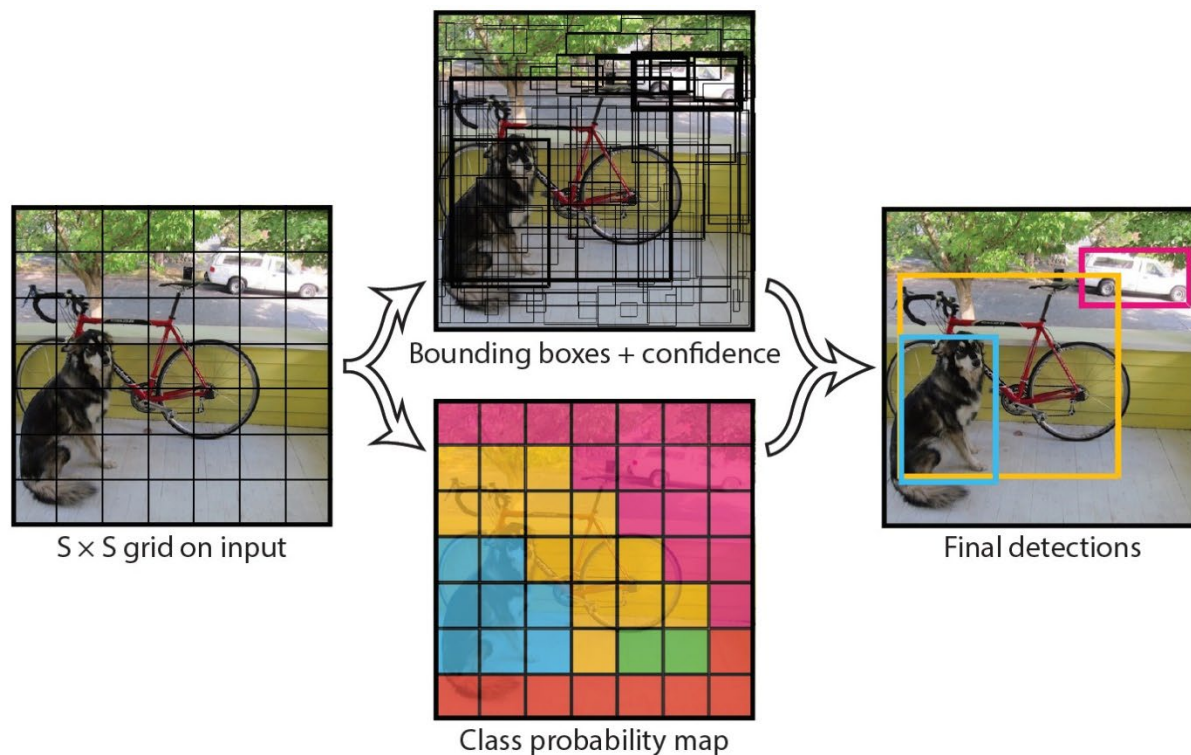
Output:
 $7 \times 7 \times (5 * B + C)$

Redmon et al, "You Only Look Once:
Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016

You Only Look Once (YOLO)

Divide the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities.

These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.



You Only Look Once (YOLO)

class confidence score = box confidence score $\times$ conditional class probability

box confidence score $\equiv P_r(object) \cdot IoU$

conditional class probability $\equiv P_r(class_i | object)$

class confidence score $\equiv P_r(class_i) \cdot IoU$

= box confidence score $\times$ conditional class probability

where

$P_r(object)$ is the probability the box contains an object.

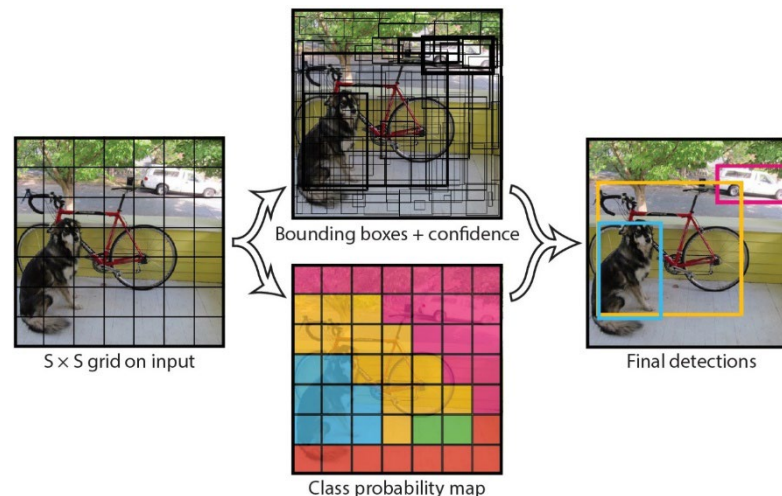
IoU is the IoU (intersection over union) between the predicted box and the ground truth.

$P_r(class_i | object)$ is the probability the object belongs to $class_i$ given an object is presence.

$P_r(class_i)$ is the probability the object belongs to $class_i$

You Only Look Once (YOLO)

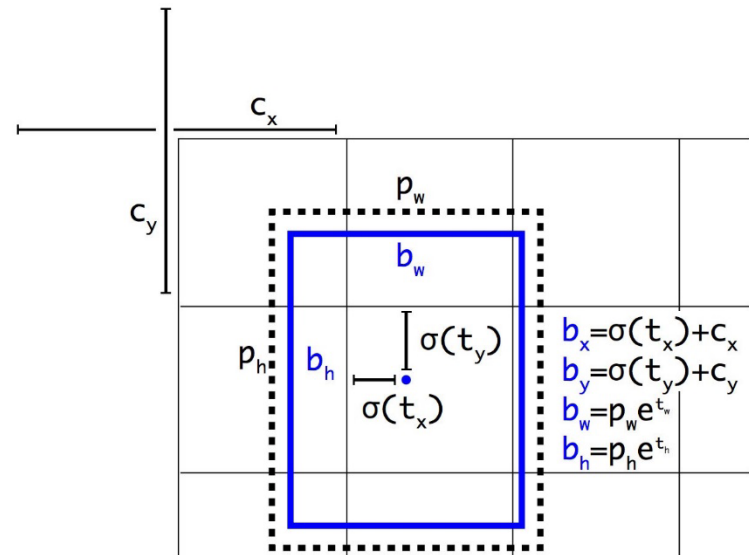
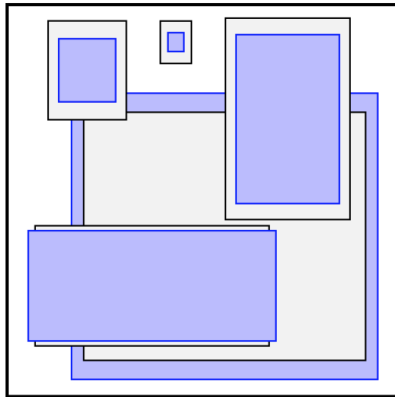
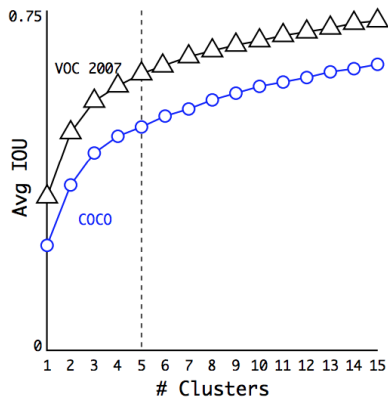
- **Fast.**
Good for real-time processing.
- **End-to-end learnable.**
Predictions (object locations and classes) are made from one **single network**.
- **Access to the entire image.**
Region proposal methods limit the classifier to the specific region.
YOLO **accesses to the whole image** in predicting boundaries.
With additional context, result in fewer false positives in background areas.
- **Spatial diversity.**
Detect one object per grid cell. It enforces spatial diversity in making predictions.



YOLOv2

- **Predetermined bounding box shape** (*anchor boxes*)
Guesses that are common for real-life objects using *k*-means clustering
⇒ Predicts **offsets** rather than bounding boxes themselves
- **Move the class prediction from the cell level to the boundary box level**
Each *bounding box* (instead of each *cell*) produces a class prediction

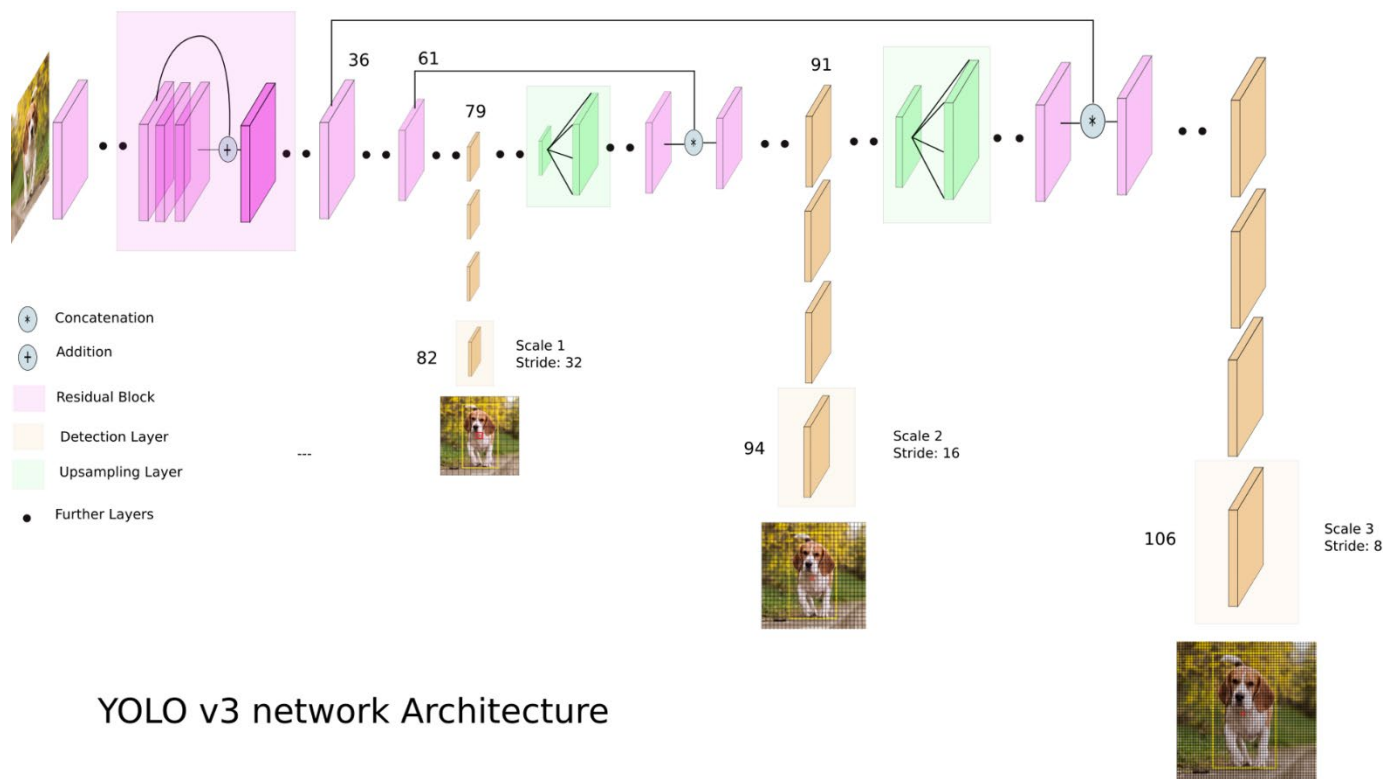
$$S \times S \times (B * 5 + C) \Rightarrow S \times S \times (B * (5 + C))$$



YOLOv3

- **Feature Pyramid Networks (FPN) like Feature Pyramid**
YOLOv3 makes predictions at 3 different scales (similar to the FPN).

$$S \times S \times (3 * (5 + C))$$



Recap

Object Detection Methods

{ Sliding Windows

Two-stage Frameworks

(High Accuracy, Slow)

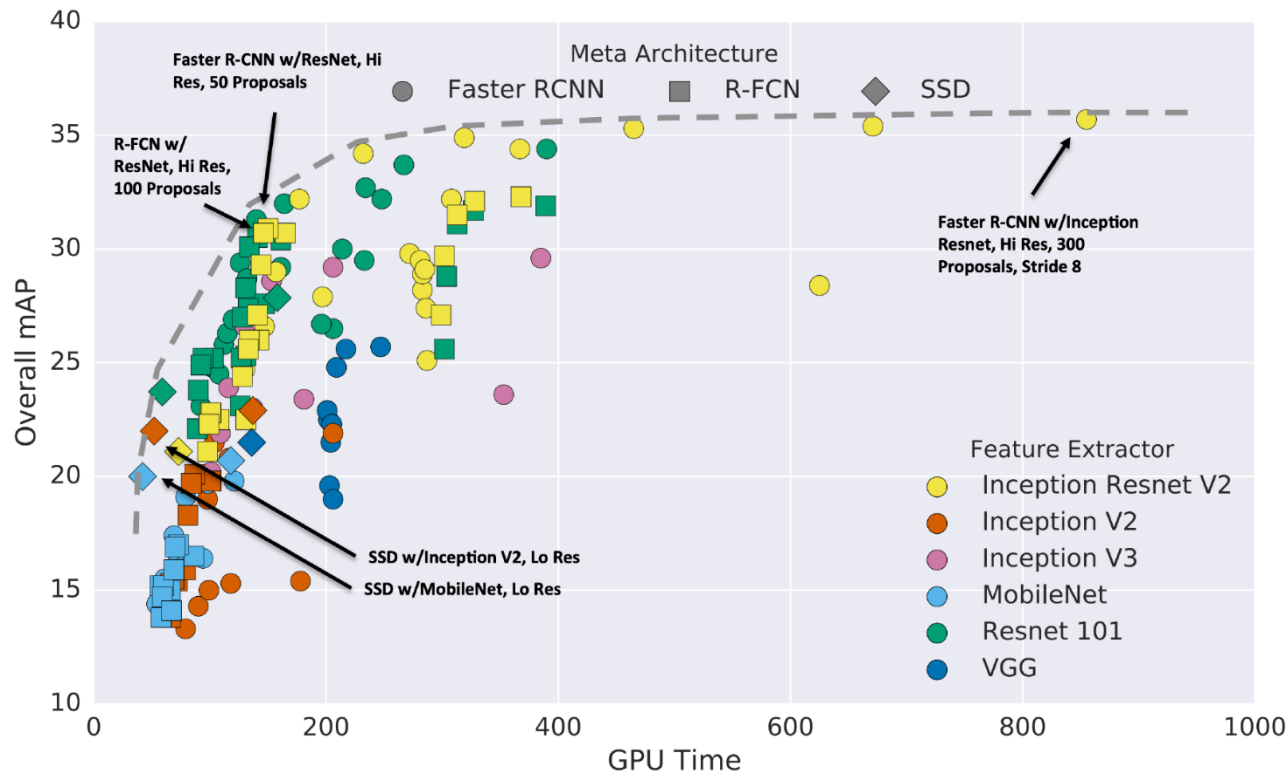
{ R-CNN
Fast R-CNN
Mask R-CNN
⋮

One-stage Frameworks

(Good Accuracy, Very Fast)

{ YOLO
YOLOv2
YOLOv3
⋮

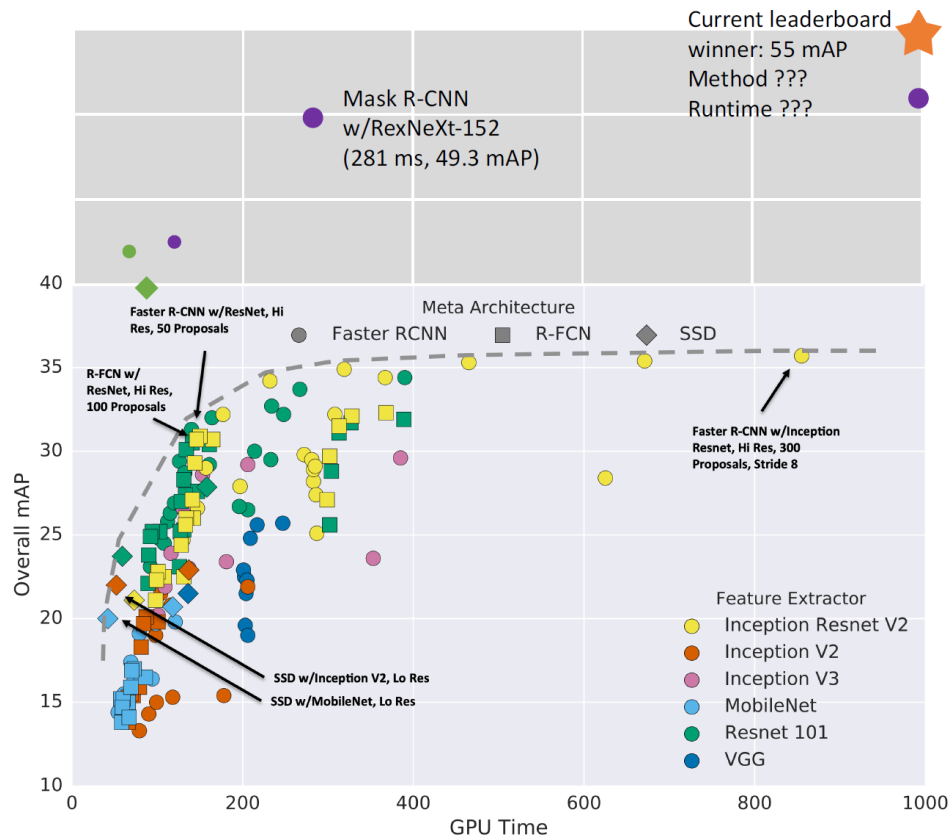
Remarks



Takeaways:

- Two stage method (Faster R-CNN) get the best accuracy, but are slower
- Single-stage methods (SSD) are much faster, but don't perform as well
- Bigger backbones improve performance, but are slower

Remarks (cont'd)



These results are a few years old ... since then GPUs have gotten faster, and we've improved performance with many tricks:

- Train longer!
- Multiscale backbone: Feature Pyramid Networks
- Better backbone: ResNeXt
- Single-Stage methods have improved
- Very big models work better
- Test-time augmentation pushes numbers up
- Big ensembles, more data, etc

What We've Covered...

- Image Segmentation
- Object Detection
 - Detection via Sliding Windows
 - Two-Stage vs. Single-Stage Detectors
 - ~~Transformer-based Detectors~~
 - ~~3D Detection~~

